

การตรวจสอบค่าความคลาดเคลื่อนในข้อมูลแบบปรับตัวได้
สำหรับการพยากรณ์ข้อมูลอนุกรมเวลา

ประกาศ กั้นจិនะ

งานวิจัยเล่มนี้เป็นส่วนหนึ่งของรายวิชา

โครงการวิจัยทางด้านวิทยาการข้อมูลเชิงสถิติ (DS4901)

ตามหลักสูตรวิทยาศาสตรบัณฑิต สาขาสถิติประยุกต์ (แขนงวิชาวิทยาการข้อมูลเชิงสถิติ)

คณะวิทยาศาสตร์และเทคโนโลยี มหาวิทยาลัยราชภัฏเชียงใหม่

ปีการศึกษา 2568

การตรวจสอบค่าความคลาดเคลื่อนในข้อมูลแบบปรับตัวได้
สำหรับการพยากรณ์ข้อมูลอนุกรมเวลา

ประกาศ กั้นจិនะ

งานวิจัยเล่มนี้เป็นส่วนหนึ่งของรายวิชา

โครงการวิจัยทางด้านวิทยาการข้อมูลเชิงสถิติ (DS4901)

ตามหลักสูตรวิทยาศาสตรบัณฑิต สาขาสถิติประยุกต์ (แขนงวิชาวิทยาการข้อมูลเชิงสถิติ)

คณะวิทยาศาสตร์และเทคโนโลยี มหาวิทยาลัยราชภัฏเชียงใหม่

ปีการศึกษา 2568

การตรวจสอบค่าความคลาดเคลื่อนในข้อมูลแบบปรับตัวได้สำหรับการพยากรณ์อนุกรมเวลา

ประกาศ กำนัน

รายงานฉบับนี้ได้รับการพิจารณาอนุมัติให้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตร
ปริญญาวิทยาศาสตรบัณฑิต สาขาสถิติประยุกต์ (แขนงวิทยาการข้อมูลเชิงสถิติ)
คณะวิทยาศาสตร์และเทคโนโลยี มหาวิทยาลัยราชภัฏเชียงใหม่

คณะกรรมการการตรวจสอบ

ลงชื่อ.....อาจารย์ที่ปรึกษา

(อาจารย์ ดร.กุลจิรา กิ่งไพร)

ลงชื่อ.....กรรมการสอบ

(อาจารย์ ดร.ปิยะชาติ เวียงนาค)

ลงชื่อ.....กรรมการสอบ

(อาจารย์ ดร.กมล สนิทธรรม)

กุมภาพันธ์ 2569

คณะวิทยาศาสตร์และเทคโนโลยี มหาวิทยาลัยราชภัฏเชียงใหม่

เรื่อง	การตรวจสอบค่าความคลาดเคลื่อนในข้อมูลแบบปรับตัวได้ สำหรับการพยากรณ์ข้อมูลอนุกรมเวลา
ผู้วิจัย	ประภากร กันจันะ
อาจารย์ที่ปรึกษาวิจัย	อาจารย์ ดร.กุลจิรา กิ่งไพร
ปีการศึกษา	2568

บทคัดย่อ

งานวิจัยนี้มีวัตถุประสงค์เพื่อพัฒนาและประเมินแนวทางการตรวจสอบความถูกต้องของแบบจำลองพยากรณ์ข้อมูลอนุกรมเวลาในสถานะที่ข้อมูลมีการเปลี่ยนแปลงของโครงสร้าง โดยประยุกต์ใช้แนวคิด Adaptive Cross-Validation ที่อิงจากการตรวจจับ Concept Drift ด้วยอัลกอริทึม Adaptive Windowing (ADWIN) เพื่อเพิ่มความน่าเชื่อถือของกระบวนการประเมินแบบจำลอง

ข้อมูลที่ใช้ในการศึกษาเป็นข้อมูลราคาหุ้นรายวันของบริษัท Meta Platforms, Inc. (META), NVIDIA Corporation (NVIDIA) และ Tesla, Inc. (TSLA) ในช่วงระยะเวลา 10 ปี ตั้งแต่ปี พ.ศ. 2558 ถึง พ.ศ. 2567 โดยใช้แบบจำลองพยากรณ์ 4 ประเภท ได้แก่ Linear Regression, Recurrent Neural Network (RNN), Long Short-Term Memory (LSTM) และ Gated Recurrent Unit (GRU) และประเมินผลด้วยค่า Mean Absolute Error (MAE) และ Root Mean Square Error (RMSE)

ผลการศึกษาพบว่าแนวทาง Adaptive Cross-Validation ที่อิงจากจุด Concept Drift ให้ผลการประเมินแบบจำลองที่มีความแม่นยำและสอดคล้องกับสภาพข้อมูลจริงมากกว่าวิธี Cross-Validation แบบดั้งเดิม อย่างมีนัยสำคัญ โดยพิจารณาจากค่าความคลาดเคลื่อน RMSE และ MAE ที่ลดลงในเกือบทุกกรณีศึกษา ตัวอย่างเช่น ในข้อมูลราคาหุ้นที่มีความผันผวนสูง ค่า RMSE เฉลี่ยของแบบจำลองภายใต้ Adaptive Cross-Validation ลดลงประมาณ 10–25% เมื่อเทียบกับ Baseline Cross-Validation ขณะที่ค่า MAE มีแนวโน้มลดลงในทิศทางเดียวกัน แสดงให้เห็นว่าการแบ่งชุดข้อมูลตามจุดเปลี่ยนของโครงสร้างข้อมูลช่วยลดอิทธิพลของข้อมูลที่ไม่สอดคล้องกับสถานการณ์ปัจจุบันได้อย่างมีประสิทธิภาพ

ผลการเปรียบเทียบพบว่า LSTM และ GRU ให้ค่าความคลาดเคลื่อนต่ำกว่า Linear Regression อย่างสม่ำเสมอเมื่อใช้ร่วมกับ Adaptive Cross-Validation โดยสรุป การคำนึงถึง Concept Drift ช่วยเพิ่มความน่าเชื่อถือของการประเมินแบบจำลอง และ Adaptive Cross-Validation เหมาะสมสำหรับข้อมูลอนุกรมเวลาที่มีการเปลี่ยนแปลงของโครงสร้างอย่างต่อเนื่อง

คำสำคัญ: Concept Drift, Adaptive Cross-Validation, GRU, RNN, LSTM, NVIDIA

กิตติกรรมประกาศ

งานวิจัยเรื่อง การตรวจสอบค่าความคลาดเคลื่อนในข้อมูลแบบปรับตัวได้สำหรับการพยากรณ์ข้อมูลอนุกรมเวลา จัดทำขึ้นเป็นส่วนหนึ่งของรายวิชาโครงการวิจัยทางด้านวิทยาการข้อมูลเชิงสถิติ (DS 4901-63) สำหรับนักศึกษาชั้นปีที่ 4 คณะวิทยาศาสตร์และเทคโนโลยี สาขาสถิติประยุกต์ (แขนงวิชาวิทยาการข้อมูลเชิงสถิติ)

งานวิจัยฉบับนี้สามารถดำเนินการจนแล้วเสร็จสมบูรณ์ได้ เนื่องจากได้รับความกรุณาอย่างยิ่งจากอาจารย์ ดร.กุลจิรา กิ่งไพร อาจารย์ที่ปรึกษา ซึ่งได้ให้คำปรึกษา คำแนะนำ ตลอดจนข้อคิดเห็นอันเป็นประโยชน์ยิ่งต่อการดำเนินงานวิจัยตั้งแต่เริ่มต้นจนสำเร็จลุล่วง ผู้วิจัยจึงขอกราบขอบพระคุณเป็นอย่างสูงไว้ ณ โอกาสนี้

นอกจากนี้ ผู้วิจัยขอขอบคุณผู้ทรงคุณวุฒิและผู้เชี่ยวชาญทุกท่านที่ได้กรุณาให้ความอนุเคราะห์ในการตรวจสอบและสนับสนุนการดำเนินงานวิจัย พร้อมทั้งให้ข้อเสนอแนะอันเป็นประโยชน์ ซึ่งมีส่วนช่วยให้ผลงานวิจัยฉบับนี้มีความถูกต้องและสมบูรณ์ยิ่งขึ้น

ท้ายที่สุด ผู้วิจัยขอขอบคุณเจ้าของเอกสาร ตำรา และงานวิจัยต่าง ๆ ที่ได้นำมาใช้อ้างอิงในการจัดทำรายงานฉบับนี้ หวังเป็นอย่างยิ่งว่างานวิจัยฉบับนี้จะเป็นประโยชน์ต่อผู้อ่าน นักศึกษา และผู้ที่สนใจศึกษาค้นคว้าในสาขาที่เกี่ยวข้อง ทั้งนี้ หากมีข้อบกพร่องหรือความผิดพลาดประการใด ผู้วิจัยขอน้อมรับไว้และขออภัยมา ณ ที่นี้

นายประภากร กันจันะ

ผู้วิจัย

สารบัญ

เรื่อง	หน้า
บทคัดย่อ	ก
กิตติกรรมประกาศ	ข
สารบัญ	ค
บทที่ 1 บทนำ	1
1.1 ที่มาและความสำคัญ	1
1.2 วัตถุประสงค์	2
1.3 สมมติฐานของงานวิจัย	2
1.4 ขอบเขตงานวิจัย	2
1.5 กรอบแนวคิดในการวิจัย	3
1.6 นิยามศัพท์	5
1.7 ประโยชน์ที่คาดว่าจะได้รับ	6
บทที่ 2 ทบทวนวรรณกรรม	7
2.1 การดริฟต์ของแนวคิด (CONCEPT DRIFT)	7
2.1.1 ลักษณะการเปลี่ยนแปลงของแนวคิด (Transition of Change)	8
2.2 CROSS-VALIDATION ใน TIME SERIES	9
2.2.1 Rolling Window	9
2.2.2 Expanding Windows	10
2.3 สถิติที่ใช้ตรวจ DRIFT	11
2.3.1 อัลกอริทึม Adaptive Windowing (ADWIN)	11
2.4 แบบจำลองพยากรณ์	12
2.4.1 Recurrent Neural Network (RNN)	12
2.4.2 Long Short-Term Memory (LSTM)	13
2.4.3 Gated Recurrent Unit (GRU)	14
2.4.4 Linear Regression	15
2.5 METRIC ในการประเมิน	16
2.5.1 ค่าเฉลี่ยความคลาดเคลื่อนสัมบูรณ์ (Mean Absolute Error: MAE)	16
2.5.2 ค่ารากที่สองของความคลาดเคลื่อนเฉลี่ยกำลังสอง (Root Mean Square Error: RMSE)	16
2.6 งานวิจัยที่เกี่ยวข้อง	16

สารบัญ (ต่อ)

เรื่อง	หน้า
บทที่ 3 วิธีดำเนินการวิจัย	20
3.1 การเก็บรวบรวมข้อมูล	20
3.2 การเตรียมข้อมูลและการสร้างตัวแปร (FEATURE ENGINEERING)	20
3.2.1. การนำเข้าข้อมูล	20
3.2.2. ทำการเช็คข้อมูลมีค่าว่าง (Missing Data) และตรวจสอบจำนวนแถวข้อมูลที่ซ้ำกัน	20
3.2.3 การวิเคราะห์ข้อมูล	21
3.2.4 เตรียมข้อมูลสำหรับการสร้างแบบจำลองการพยากรณ์	21
3.3 การกำหนดตัวแปรคุณลักษณะ (X) และตัวแปรเป้าหมาย (Y)	21
3.4 การตรวจจับ CONCEPT DRIFT (DRIFT DETECTION)	22
3.4.1 หลักการของ ADWIN	22
3.4.2 ขั้นตอนการตรวจจับ	22
3.5 แบบจำลองที่ใช้ในการทำนาย	22
3.5.1 โครงสร้างแบบจำลอง ใช้ 3 ประเภท	22
3.5.2 การเตรียมข้อมูลสำหรับ RNN	23
3.5.3 พารามิเตอร์การฝึก	23
3.5.4 Linear Regression	23
3.6 กลยุทธ์การตรวจสอบความถูกต้อง (CROSS-VALIDATION STRATEGIES)	23
3.6.1 Adaptive Cross-Validation	23
3.6.2 Baseline Cross-Validation	24
3.7 การวัดประสิทธิภาพของโมเดล	24
3.8 ฝั่งงานกระบวนการตรวจจับการเปลี่ยนแปลงแบบปรับตัว	24
บทที่ 4 ผลการศึกษา	27
4.1 การตรวจจับ CONCEPT DRIFT	27
4.2 ผลการทดลองด้วย ADAPTIVE และ BASELINE CROSS-VALIDATION	29
4.2.1 ผลการทดลองด้วย Adaptive และ Baseline Cross-Validation ของหุ้น META	29
4.2.2 ผลการทดลองด้วย Adaptive และ Baseline Cross-Validation ของหุ้น NVIDIA	32
4.2.3 ผลการทดลองด้วย Adaptive และ Baseline Cross-Validation ของหุ้น TSLA	35

สารบัญ (ต่อ)

เรื่อง	หน้า
4.3 สรุปผลการวิเคราะห์	38
4.3.1 ผลลัพธ์ภายใต้ Adaptive Cross-Validation	38
4.3.2 ผลลัพธ์ภายใต้ Baseline Cross-Validation	39
4.3.3 การเปรียบเทียบเชิงปริมาณและเชิงเหตุผล	39
4.4 สรุปผลการวิเคราะห์	40
บทที่ 5 สรุป อภิปรายผล และข้อเสนอแนะ	41
5.1 สรุปผลการวิจัย	41
5.2 อภิปรายผลการวิจัย	42
5.3 ข้อเสนอแนะ	42
บรรณานุกรม	43
ภาคผนวก	46

สารบัญภาพ

ภาพ	หน้า
ภาพที่ 1 กรอบแนวคิดในการวิจัย	3
ภาพที่ 2 สรุปแผนภาพทั่วไปของวิธีการตรวจจับการเปลี่ยนแปลงแนวคิด	8
ภาพที่ 3 การจำแนกประเภทของการเปลี่ยนแปลงแนวคิดตามลักษณะของรูปแบบการปรากฏ	9
ภาพที่ 4 หลักการทำงานของ ROLLING WINDOW	10
ภาพที่ 5 หลักการทำงานของ EXPANDING WINDOW	11
ภาพที่ 6 โครงสร้างและหลักการทำงานของ RNN	13
ภาพที่ 7 โครงสร้างและหลักการทำงานของ LSTM	14
ภาพที่ 8 โครงสร้างและหลักการทำงานของ GRU	15
ภาพที่ 9 ผังงานแสดงกระบวนการ ADAPTIVE DRIFT DETECTION	24
ภาพที่ 10 การตรวจจับ CONCEPT DRIFT ของข้อมูลราคาหุ้น META	28
ภาพที่ 11 การตรวจจับ CONCEPT DRIFT ของข้อมูลราคาหุ้น NVIDIA	28
ภาพที่ 12 การตรวจจับ CONCEPT DRIFT ของข้อมูลราคาหุ้น TSLA	28
ภาพที่ 13 การเปรียบเทียบค่าเฉลี่ยของ RMSE ในแต่ละโมเดลของหุ้น META	31
ภาพที่ 14 การเปรียบเทียบค่าเฉลี่ยของ MAE ในแต่ละโมเดลของหุ้น META	31
ภาพที่ 15 การเปรียบเทียบค่าเฉลี่ยของ RMSE ในแต่ละโมเดลของหุ้น NVIDIA	34
ภาพที่ 16 การเปรียบเทียบค่าเฉลี่ยของ MAE ในแต่ละโมเดลของหุ้น NVIDIA	34
ภาพที่ 17 การเปรียบเทียบค่าเฉลี่ยของ RMSE ในแต่ละโมเดลของหุ้น TSLA	37
ภาพที่ 18 การเปรียบเทียบค่าเฉลี่ยของ MAE ในแต่ละโมเดลของหุ้น TSLA	37
ภาพที่ 19 การนำเข้าข้อมูลของ META STOCK	47
ภาพที่ 20 รายละเอียดของชุดข้อมูลดั้งเดิมของ META STOCK	47
ภาพที่ 21 ตรวจสอบจำนวนค่าว่าง และจำนวนแถวข้อมูลที่ซ้ำกันของ META STOCK	47
ภาพที่ 22 รายละเอียดค่าทางสถิติเบื้องต้นของ META STOCK	48
ภาพที่ 23 ชุดข้อมูลคงเหลือหลังจากเพิ่ม FEATURE ENGINEERING ของ META STOCK	48
ภาพที่ 24 รายละเอียดค่าทางสถิติเบื้องต้นหลังจากเพิ่ม FEATURE ENGINEERING ของ META STOCK	48
ภาพที่ 25 การนำเข้าข้อมูลของ NVIDIA STOCK	49
ภาพที่ 26 รายละเอียดของชุดข้อมูลดั้งเดิมของ NVIDIA STOCK	49
ภาพที่ 27 ตรวจสอบจำนวนค่าว่าง และจำนวนแถวข้อมูลที่ซ้ำกันของ NVIDIA STOCK	49
ภาพที่ 28 รายละเอียดค่าทางสถิติเบื้องต้นของ NVIDIA STOCK	50
ภาพที่ 29 ชุดข้อมูลคงเหลือหลังจากเพิ่ม FEATURE ENGINEERING	50

สารบัญภาพ (ต่อ)

ภาพ	หน้า
ภาพที่ 30 รายละเอียดค่าทางสถิติเบื้องต้นหลังจากเพิ่ม FEATURE ENGINEERING	50
ภาพที่ 31 การนำเข้าข้อมูลของ TSLA STOCK	51
ภาพที่ 32 รายละเอียดของชุดข้อมูลดั้งเดิมของ TSLA STOCK	51
ภาพที่ 33 ตรวจสอบจำนวนค่าว่าง และจำนวนแถวข้อมูลที่ซ้ำกันของ TSLA STOCK	51
ภาพที่ 34 รายละเอียดค่าทางสถิติเบื้องต้นของ TSLA STOCK	52
ภาพที่ 35 ชุดข้อมูลคงเหลือหลังจากเพิ่ม FEATURE ENGINEERING ของ TSLA STOCK	52
ภาพที่ 36 รายละเอียดค่าทางสถิติเบื้องต้นหลังจากเพิ่ม FEATURE ENGINEERING ของ TSLA STOCK	52

สารบัญตาราง

ตาราง	หน้า
ตารางที่ 1 ผลคลาดเคลื่อนแต่ละโพลด์ของ ADAPTIVE CV (META)	29
ตารางที่ 2 ผลคลาดเคลื่อนแต่ละโพลด์ของ BASELINE-5 CV (META)	29
ตารางที่ 3 ผลคลาดเคลื่อนแต่ละโพลด์ของ BASELINE-10 CV (META)	30
ตารางที่ 4 ผลคลาดเคลื่อนแต่ละโพลด์ของ BASELINE CV (เท่ากับจำนวนโพลด์ ADAPTIVE) ของ META	30
ตารางที่ 5 ผลคลาดเคลื่อนแต่ละโพลด์ของ ADAPTIVE CV (NVIDIA)	32
ตารางที่ 6 ผลคลาดเคลื่อนแต่ละโพลด์ของ BASELINE-5 CV (NVIDIA)	32
ตารางที่ 7 ผลคลาดเคลื่อนแต่ละโพลด์ของ BASELINE-10 CV (NVIDIA)	33
ตารางที่ 8 ผลคลาดเคลื่อนแต่ละโพลด์ของ BASELINE CV (เท่ากับจำนวนโพลด์ ADAPTIVE) ของ NVIDIA	33
ตารางที่ 9 ผลคลาดเคลื่อนแต่ละโพลด์ของ ADAPTIVE CV (TESLA)	35
ตารางที่ 10 ผลคลาดเคลื่อนแต่ละโพลด์ของ BASELINE-5 CV (TESLA)	35
ตารางที่ 11 ผลคลาดเคลื่อนแต่ละโพลด์ของ BASELINE-10 CV (TESLA)	36
ตารางที่ 12 ผลคลาดเคลื่อนแต่ละโพลด์ของ BASELINE CV (เท่ากับจำนวนโพลด์ ADAPTIVE) ของ TESLA	36

บทที่ 1

บทนำ

1.1 ที่มาและความสำคัญ

ในอดีต การประเมินความถูกต้องของแบบจำลองสำหรับข้อมูลอนุกรมเวลา มักอาศัยวิธีการตรวจสอบความถูกต้อง (Cross-Validation) ที่มีรูปแบบคงที่ เช่น การเลื่อนหน้าต่าง (Rolling Window) และการขยายหน้าต่าง (Expanding Window) วิธีเหล่านี้ตั้งอยู่บนสมมติฐานสำคัญว่าการกระจายของข้อมูลในแต่ละช่วงเวลามีความคงที่ (Stationary) จึงสามารถใช้ข้อมูลในอดีตเพื่อประเมินประสิทธิภาพแบบจำลองและคาดการณ์อนาคตได้อย่างแม่นยำ

อย่างไรก็ตาม ข้อมูลอนุกรมเวลาจริง เช่น ราคาหุ้น พฤติกรรมผู้ใช้บริการ หรือข้อมูลจากเซนเซอร์ในระบบอุตสาหกรรม กลับมีลักษณะเปลี่ยนแปลงไปตามกาลเวลา หรือที่เรียกว่า Concept Drift ซึ่งหมายถึงการเปลี่ยนแปลงของโครงสร้างหรือความสัมพันธ์ในข้อมูล ทำให้แบบจำลองที่สร้างจากข้อมูลในอดีตอาจสูญเสียความแม่นยำเมื่อใช้กับข้อมูลใหม่ การเพิกเฉยต่อปรากฏการณ์นี้ในกระบวนการประเมินแบบจำลองอาจนำไปสู่การเลือกโมเดลหรือพารามิเตอร์ที่ไม่เหมาะสม และทำให้ผลการพยากรณ์ในสภาพแวดล้อมจริงมีความคลาดเคลื่อนสูง

เพื่อรับมือกับปัญหาดังกล่าว มีงานวิจัยหลายชิ้นที่มุ่งพัฒนาวิธีตรวจจับและจัดการ Concept Drift เช่น งานวิจัยของ Tsymbal (2004) ได้เสนอสามแนวทางหลักในการรับมือ Concept Drift ได้แก่ 1) Instance Selection คือการเลือกใช้เฉพาะข้อมูลย้อนหลังที่ใกล้เคียงกับสถานการณ์ปัจจุบันเพื่อลดอิทธิพลของข้อมูลเก่า 2) Instance Weighting คือการให้น้ำหนักข้อมูลต่างกัน โดยข้อมูลที่ใกล้ปัจจุบันมีน้ำหนักมากกว่า เพื่อบ่งชี้ความสำคัญของข้อมูลล่าสุด และ 3) Ensemble Learning คือการใช้หลายโมเดลที่ฝึกจากข้อมูลต่างช่วงเวลาหรือเงื่อนไข แล้วรวมผลพยากรณ์เพื่อเพิ่มความยืดหยุ่น แนวทางเหล่านี้มีจุดมุ่งหมายร่วมกัน คือปรับแบบจำลองให้สอดคล้องกับข้อมูลที่เปลี่ยนแปลง และรักษาความแม่นยำของการพยากรณ์ งานวิจัยของ Barros และ Santos (2018) ได้เปรียบเทียบวิธีตรวจจับ Concept Drift จำนวน 14 วิธี ทั้งใน Drift แบบฉับพลันและค่อยเป็นค่อยไป โดยใช้ตัวจำแนก Naive Bayes และ Hoeffding Tree พบว่าวิธี RDDM และ HDDM_A ให้ค่า Accuracy สูงสุด (88–92% สำหรับ Abrupt Drift และ 86–90% สำหรับ Gradual Drift) และมีอัตราแจ้งเตือนผิดต่ำ การวิเคราะห์เพิ่มเติมด้วย Nemenyi test ยังยืนยันว่าประสิทธิภาพของวิธีเหล่านี้แตกต่างจากวิธีอื่นอย่างมีนัยสำคัญ

นอกจากนี้ งานวิจัยของ Zenisek, Holzinger และ Affenzeller (2019) ได้นำแนวคิดการตรวจจับ Drift มาประยุกต์ใช้ในงาน Predictive Maintenance โดยใช้โมเดลพยากรณ์ เช่น Linear Regression และ Random Forest Regression เพื่อติดตามความแตกต่างระหว่างค่าพยากรณ์กับค่าจริงของข้อมูลเซนเซอร์ เมื่อความคลาดเคลื่อนเกินเกณฑ์ที่กำหนดจะถูกตีความว่าเป็นสัญญาณ Drift หรือความผิดปกติในเครื่องจักร

ผลลัพธ์แสดงให้เห็นว่าสามารถตรวจพบความผิดปกติได้ล่วงหน้าหลายชั่วโมงก่อนเกิดปัญหาจริง ด้วยความแม่นยำสูงถึง 92–94% และมีอัตราแจ้งเตือนผิดพลาดต่ำกว่า 5%

อย่างไรก็ดี งานวิจัยส่วนใหญ่ก็ให้ความสำคัญกับการตรวจจับ Concept Drift โดยตรง มากกว่า การนำกระบวนการตรวจสอบความถูกต้องของแบบจำลองมาผสานกับการตรวจจับการเปลี่ยนแปลงของข้อมูล อย่างเป็นระบบ ดังนั้น งานวิจัยนี้จึงมุ่งพัฒนาแนวทางการตรวจสอบค่าความคลาดเคลื่อนของข้อมูล แบบปรับตัวได้ (Adaptive Error Monitoring) ที่เชื่อมโยงกับขั้นตอนการประเมินแบบจำลองสำหรับข้อมูล อนุกรมเวลา เพื่อให้การแบ่งชุดข้อมูลฝึกและทดสอบมีความเหมาะสมกับสภาพข้อมูลที่เปลี่ยนแปลงไป ตามเวลา ลดความเสี่ยงในการเลือกแบบจำลองที่ไม่สอดคล้องกับการใช้งานจริง และเพิ่มความแม่นยำ ของการพยากรณ์ในสถานการณ์ที่ข้อมูลมีการเปลี่ยนแปลงอยู่เสมอ

1.2 วัตถุประสงค์

- พัฒนารูปแบบการ Cross-Validation แบบใหม่สำหรับข้อมูลอนุกรมเวลาที่สามารถตรวจจับ Concept Drift โดยใช้อัลกอริทึมที่ใช้ตรวจจับการเปลี่ยนแปลงของข้อมูลใน Data Stream ได้แก่ ADWIN (Adaptive Windowing)
- กำหนดช่วงการแบ่งกลุ่มข้อมูลฝึกและทดสอบโดยอิงจากจุดที่ตรวจพบ Drift แทน การแบ่งตามช่วงเวลาแบบคงที่
- เพิ่มความน่าเชื่อถือในการประเมินแบบจำลอง โดยให้ชุดข้อมูลฝึกและทดสอบสะท้อนสถานการณ์จริง ที่อาจเกิดขึ้นในอนาคต

1.3 สมมติฐานของงานวิจัย

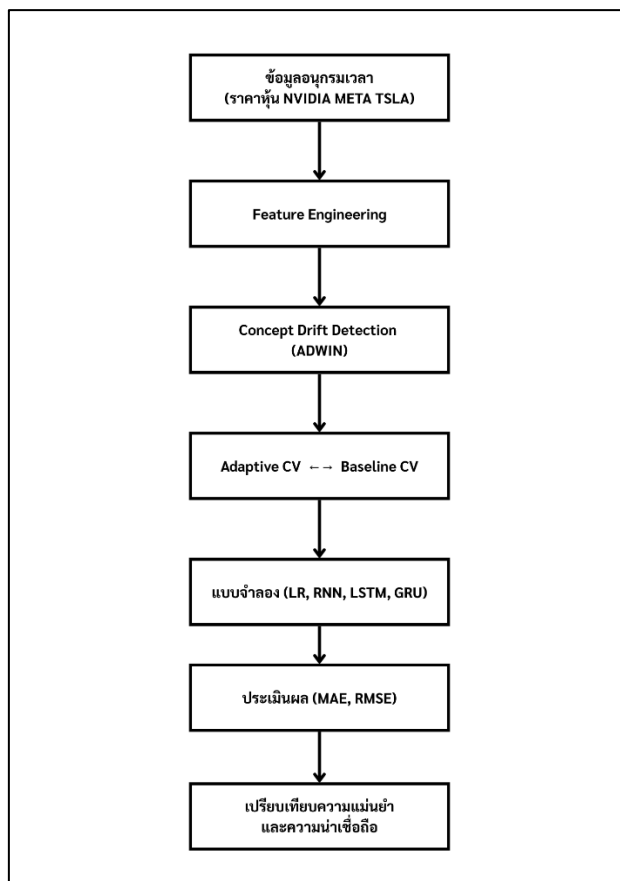
- การใช้ Adaptive Cross-Validation ที่อิงจากจุด Drift จะให้ผลการประเมินแบบจำลองที่แม่นยำ และน่าเชื่อถือกว่าวิธีการ Cross-Validation แบบดั้งเดิม

1.4 ขอบเขตงานวิจัย

- ข้อมูลที่ใช้:
 - ราคาหุ้นรายวันของบริษัท Meta Platforms, Inc. ซึ่งเป็นบริษัทในกลุ่มอุตสาหกรรม Interactive Media & Services / Social Media
 - ราคาหุ้นรายวันของบริษัท NVIDIA Corporation ในกลุ่มอุตสาหกรรม Semiconductors
 - ราคาหุ้นรายวันของบริษัท Tesla, Inc. ซึ่งเป็นบริษัทในกลุ่มอุตสาหกรรม Automobile Manufacturers (EVs)
- ช่วงเวลา: 2 มกราคม 2558 – 31 ธันวาคม 2567
- แหล่งข้อมูล: Yahoo Finance (ดึงข้อมูลด้วยไลบรารี yfinance บนภาษา Python)

- ตัวแปรสำคัญ: วันที่ (Date), ราคาปิด (Close), ราคาสูงสุด (High), ราคาต่ำสุด (Low), ปริมาณการซื้อขาย (Volume)

1.5 กรอบแนวคิดในการวิจัย



ภาพที่ 1 กรอบแนวคิดในการวิจัย

1.5.1 ข้อมูลอนุกรมเวลา (Time Series Data)

เริ่มต้นจากข้อมูลดิบที่เป็นข้อมูลอนุกรมเวลา ซึ่งในที่นี้คือ ราคาหุ้นของ NVIDIA META และ TSLA ข้อมูลประเภทนี้จะมีการเรียงลำดับตามช่วงเวลาที่เกิดขึ้น

1.5.2 Feature Engineering

ขั้นตอนนี้เป็นการ สร้างฟีเจอร์หรือคุณลักษณะใหม่ๆ จากข้อมูลดิบเพื่อนำไปใช้ในการสร้างโมเดล เพื่อสามารถช่วยให้โมเดลเรียนรู้และพยากรณ์ได้ดีขึ้น

1.5.3 Concept Drift Detection (ADWIN)

Concept drift คือปรากฏการณ์ที่ความสัมพันธ์ระหว่างข้อมูลและตัวแปรเป้าหมายเปลี่ยนแปลงไปตามกาลเวลา ซึ่งในบริบทของราคาหุ้นก็คือ พฤติกรรมของตลาดหุ้นที่อาจเปลี่ยนแปลงไปจากเดิม การตรวจจับการเปลี่ยนแปลงนี้จึงสำคัญมาก

ADWIN (Adaptive Windowing) เป็นอัลกอริทึมที่ใช้ในการตรวจจับ concept drift โดยหลักการการทำงานคือการ เปรียบเทียบค่าเฉลี่ยของข้อมูลในช่วงเวลาที่ต่างกัน เมื่อพบว่าค่าเฉลี่ยแตกต่างกันอย่างมีนัยสำคัญ ADWIN จะส่งสัญญาณว่าเกิดการเปลี่ยนแปลงขึ้น และแนะนำให้ปรับปรุงโมเดลใหม่

1.5.4 การเปรียบเทียบประสิทธิภาพของโมเดล สองประเภท คือ

- Adaptive CV (Cross-Validation): เป็นการประเมินผลโมเดลที่ ปรับตัวตามการเปลี่ยนแปลงของข้อมูล โดยอาจจะมีการฝึกฝนโมเดลใหม่เมื่อตรวจพบ Concept Drift
- Baseline CV: เป็นการประเมินผลโมเดลพื้นฐานที่ไม่ได้มีการปรับตัว

การเปรียบเทียบนี้จะทำให้เห็นว่าการปรับตัวของโมเดลนั้นช่วยให้การพยากรณ์ดีขึ้นจริงหรือไม่

1.5.5 แบบจำลอง (Models)

นี่คือส่วนที่ใช้ สร้างโมเดลเพื่อพยากรณ์ราคาหุ้น ซึ่งในแผนภาพได้ระบุแบบจำลองที่นิยมใช้กับข้อมูลอนุกรมเวลาไว้ 4 แบบ ได้แก่

- LR (Linear Regression): เป็นแบบจำลองเชิงเส้นที่ง่ายที่สุด
- RNN (Recurrent Neural Network): เป็นโครงข่ายประสาทเทียมที่ออกแบบมาเพื่อจัดการกับข้อมูลที่มีลำดับ (Sequential Data)
- LSTM (Long Short-Term Memory): เป็น RNN ชนิดหนึ่งที่สามารถแก้ไขปัญหาน Vanishing Gradient และสามารถจดจำข้อมูลที่สำคัญในระยะยาวได้ดี
- GRU (Gated Recurrent Unit): คล้ายกับ LSTM แต่มีโครงสร้างที่เรียบง่ายกว่าและใช้ทรัพยากรน้อยกว่า

1.5.6 ประเมินผล (Evaluation)

หลังจากที่สร้างและทดสอบโมเดลแล้ว ก็จะมีการ ประเมินประสิทธิภาพของโมเดล โดยใช้ค่าความคลาดเคลื่อนต่างๆ เช่น

- MAE (Mean Absolute Error): ค่าเฉลี่ยของผลต่างสัมบูรณ์ระหว่างค่าที่พยากรณ์กับค่าจริง
- RMSE (Root Mean Square Error): รากที่สองของค่าเฉลี่ยกำลังสองของผลต่าง ซึ่งจะให้ความสำคัญกับค่าความคลาดเคลื่อนที่มากเป็นพิเศษ

1.5.7 เปรียบเทียบความแม่นยำและความน่าเชื่อถือ

ขั้นตอนสุดท้ายคือการ นำผลการประเมินมาเปรียบเทียบกัน เพื่อดูว่าแบบจำลองแต่ละแบบ มีความแม่นยำ (Accuracy) และความน่าเชื่อถือ (Reliability) มากน้อยแค่ไหน ซึ่งจะช่วยให้ตัดสินใจได้ว่าแบบจำลองใดเหมาะสมที่สุดสำหรับการนำไปใช้งานจริง

การที่กระบวนการนี้มีการใช้ Concept Drift Detection (ADWIN) เข้ามาด้วย แสดงให้เห็นว่าระบบนี้ ถูกออกแบบมาให้ มีความสามารถในการเรียนรู้และปรับตัว ได้อย่างต่อเนื่องตามสถานะของตลาดหุ้นที่เปลี่ยนแปลงไปได้ ซึ่งเป็นสิ่งสำคัญสำหรับงานพยากรณ์ในโลกความเป็นจริง

1.6 นิยามศัพท์

- Concept Drift คือปรากฏการณ์ที่ความสัมพันธ์ระหว่างตัวแปรต้น (Input) และตัวแปรตาม (Target) มีการเปลี่ยนแปลงไปตามกาลเวลา ทำให้แบบจำลองที่สร้างขึ้นจากข้อมูลในอดีตสูญเสียความแม่นยำเมื่อใช้กับข้อมูลใหม่ ตัวอย่างเช่น การเปลี่ยนแปลงพฤติกรรมตลาดหุ้นหรือพฤติกรรมผู้บริโภค
- Cross-Validation (การตรวจสอบความถูกต้องแบบไขว้) คือวิธีการแบ่งชุดข้อมูลออกเป็นส่วนย่อย ๆ เพื่อใช้ในการฝึก (Training) และทดสอบ (Testing) แบบจำลอง โดยมีจุดมุ่งหมายเพื่อประเมินประสิทธิภาพและความน่าเชื่อถือของโมเดลในสถานะข้อมูลที่แตกต่างกัน
- Adaptive Cross-Validation (Adaptive CV) คือวิธีการ Cross-Validation ที่ปรับการแบ่งข้อมูลตามจุดเปลี่ยนแปลง (Drift Point) ที่ตรวจพบ แทนการแบ่งตามช่วงเวลาแบบคงที่ ช่วยให้การประเมินแบบจำลองสะท้อนสภาพข้อมูลจริงมากยิ่งขึ้น
- Baseline Cross-Validation (Baseline CV) คือวิธีการ Cross-Validation แบบดั้งเดิมที่กำหนดการแบ่งข้อมูลตามลำดับเวลา โดยไม่คำนึงถึงการเกิด Drift ของข้อมูล
- ADWIN (Adaptive Windowing Algorithm) คืออัลกอริทึมที่ใช้ตรวจจับการเปลี่ยนแปลงของข้อมูลแบบต่อเนื่อง (Data Stream) โดยเปรียบเทียบค่าเฉลี่ยของข้อมูลในหน้าต่างเวลา (Window) ที่แตกต่างกัน หากพบความแตกต่างอย่างมีนัยสำคัญจะระบุว่าเกิด Concept Drift

- RNN (Recurrent Neural Network) คือโครงข่ายประสาทเทียมที่ออกแบบมาเพื่อจัดการข้อมูลที่มีลำดับเวลา สามารถจดจำสถานะก่อนหน้าเพื่อใช้ทำนายค่าถัดไป แต่มีข้อจำกัดเมื่อข้อมูลมีระยะยาวเนื่องจากปัญหา Vanishing Gradient
- LSTM (Long Short-Term Memory Network) คือโครงข่ายประสาทเทียมที่พัฒนาต่อยอดจาก RNN โดยเพิ่มกลไก "Gate" เพื่อแก้ปัญหาการลืมข้อมูลระยะยาว ทำให้สามารถจดจำและเรียนรู้จากข้อมูลในระยะเวลาที่ยาวขึ้นได้ดีกว่า RNN ปกติ
- GRU (Gated Recurrent Unit) คือโครงข่ายประสาทเทียมเชิงลำดับที่คล้ายกับ LSTM แต่มีโครงสร้างที่เรียบง่ายกว่า ใช้พารามิเตอร์น้อยลงแต่ยังคงประสิทธิภาพในการจัดการกับข้อมูลที่มีลำดับเวลาได้ดี
- Linear Regression (การถดถอยเชิงเส้น) คือแบบจำลองทางสถิติที่ใช้ศึกษาความสัมพันธ์เชิงเส้นระหว่างตัวแปรอิสระ (Independent Variable) และตัวแปรตาม (Dependent Variable) โดยสมมติให้ความสัมพันธ์อยู่ในรูปสมการเส้นตรง
- MAE (Mean Absolute Error) คือค่าความคลาดเคลื่อนเฉลี่ยแบบสัมบูรณ์ระหว่างค่าที่โมเดลพยากรณ์กับค่าจริง
- RMSE (Root Mean Square Error) คือค่ารากที่สองของความคลาดเคลื่อนเฉลี่ยกำลังสอง ใช้วัดความแตกต่างระหว่างค่าที่พยากรณ์และค่าจริง
- Feature Engineering (การสร้างคุณลักษณะ) คือกระบวนการสร้างหรือแปลงข้อมูลดิบให้เป็นตัวแปรใหม่ (Feature) เพื่อเพิ่มประสิทธิภาพของแบบจำลอง ตัวอย่างเช่น Return, Volatility, Volume_Log และ Return_Volume ในงานวิจัยนี้

1.7 ประโยชน์ที่คาดว่าจะได้รับ

- ได้วิธีการประเมินโมเดลสำหรับข้อมูลอนุกรมเวลาที่เหมาะสมกับสถานการณ์ที่มีการเปลี่ยนแปลงของข้อมูล
- เพิ่มความน่าเชื่อถือของการเลือกแบบจำลองและพารามิเตอร์ในงานพยากรณ์
- สามารถประยุกต์ใช้วิธี Adaptive CV กับข้อมูลอนุกรมเวลาประเภทอื่น เช่น พยากรณ์พลังงาน ตรวจสอบความผิดปกติของเครื่องจักร หรือการวิเคราะห์พฤติกรรมผู้ใช้งานระบบออนไลน์

บทที่ 2

ทบทวนวรรณกรรม

ในการวิจัยครั้งนี้ ผู้วิจัยได้ศึกษาเอกสารและงานวิจัยที่เกี่ยวข้อง และได้นำเสนอตาม หัวข้อต่อไปนี้

1. การดริฟต์ของแนวคิด (Concept Drift)
2. Cross-Validation ใน Time Series
3. สถิติที่ใช้ตรวจ Drift
4. แบบจำลองพยากรณ์
5. Metric ในการประเมิน
6. งานวิจัยที่เกี่ยวข้อง

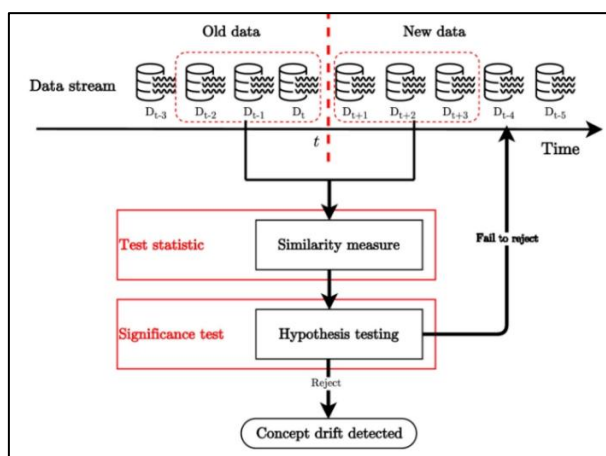
2.1 การดริฟต์ของแนวคิด (Concept Drift)

การตรวจจับการเปลี่ยนแปลงแนวคิด (Concept Drift Detection) เป็นองค์ประกอบหนึ่งของกรอบการจัดการการเปลี่ยนแปลงแนวคิด ซึ่งมีหน้าที่กระตุ้นกลไกการปรับตัวต่อการเปลี่ยนแปลงแนวคิด โดยกลไกนี้จะตอบสนองต่อความเปลี่ยนแปลงที่เกิดขึ้นในกระแสข้อมูล (Lu และคณะ 2019) ต่อจากนั้นระบบจะทำการอัปเดตความรู้เดิมและปรับปรุงแบบจำลองการเรียนรู้ให้สามารถตอบสนองต่อการเปลี่ยนแปลงได้อย่างเหมาะสม อย่างไรก็ตาม คำว่า เสถียรภาพ (Stability) หมายถึงการคงไว้ซึ่งความรู้ที่ยังมีความเกี่ยวข้องและอาจเกิดซ้ำได้ ในขณะที่ ความยืดหยุ่น (Plasticity) หมายถึงการแทนที่ความรู้เดิมที่ล้าสมัยด้วยประสบการณ์ใหม่

ในทางอุดมคติ กลยุทธ์ในการจัดการกับการเปลี่ยนแปลงแนวคิดควรสามารถสร้างสมดุลระหว่างเสถียรภาพและความยืดหยุ่นได้ Elwell และ Polikar (2011) ได้กล่าวไว้ว่า กลยุทธ์การปรับตัวต่อการเปลี่ยนแปลงแนวคิดประเภทนี้เรียกว่า การปรับตัวโดยมีข้อมูลแจ้งล่วงหน้า (Informed Adaptation) หรือที่เรียกอีกอย่างว่า แนวทางเชิงรุก (Active Approach) ซึ่งจะถูกระตุ้นเมื่อมีการตรวจพบการเปลี่ยนแปลงเพื่อปรับปรุงแบบจำลองให้ทันสมัย อีกกลยุทธ์หนึ่งคือ การปรับตัวแบบไม่รู้ล่วงหน้า (Blind Adaptation) หรือแนวทางเชิงรับ (Passive Approach) ซึ่งแบบจำลองจะถูกอัปเดตอย่างต่อเนื่องทุกครั้งที่ได้รับข้อมูลใหม่โดยไม่ตรวจสอบว่ามีการเปลี่ยนแปลงเกิดขึ้นหรือไม่ (Zhong, Yang, Zhang, Li และRen, 2021)

วิธีการตรวจจับการเปลี่ยนแปลงแนวคิดโดยทั่วไปจะใช้สถิติเชิงทดสอบในการตรวจสอบกระแสข้อมูลและคำนวณค่าความคล้ายคลึงกันระหว่างข้อมูลเก่าและข้อมูลใหม่เพื่อวิเคราะห์ว่ามีการเปลี่ยนแปลงของแนวคิดหรือไม่ จากนั้นค่าความคล้ายคลึงกันนี้จะถูกเปรียบเทียบกับค่าขีดจำกัดที่กำหนดไว้ล่วงหน้าเพื่อตรวจสอบขนาดของการเปลี่ยนแปลง (Dries และ Rückert , 2009) โดยได้รับแรงบันดาลใจจาก ภาพที่ 2 สรุปแผนภาพทั่วไปของวิธีการตรวจจับการเปลี่ยนแปลงแนวคิด ในแผนภาพดังกล่าว สมมติฐานว่าง (Null Hypothesis) คือค่าทางสถิติจากการทดสอบจะไม่แสดงความแตกต่างอย่างมีนัยสำคัญ

ระหว่างข้อมูลเก่าและข้อมูลใหม่ กล่าวคือ ไม่มีการตรวจพบการเปลี่ยนแปลงแนวคิด หากไม่สามารถปฏิเสธสมมติฐานว่างได้ ระบบจะยังคงใช้แบบจำลองปัจจุบันต่อไป และเลื่อนหน้าต่างข้อมูลเพื่อรอข้อมูลใหม่จากกระแสข้อมูลถัดไป



ภาพที่ 2 สรุปแผนภาพทั่วไปของวิธีการตรวจจับการเปลี่ยนแปลงแนวคิด

ที่มา: https://www.researchgate.net/figure/Common-examples-of-drift-detection-Adapted-with-permission-from-reference-7_fig2_373610141 สืบค้นวันที่ 17 กรกฎาคม 2568

2.1.1 ลักษณะการเปลี่ยนแปลงของแนวคิด (Transition of Change)

การจัดประเภทนี้มีจุดประสงค์เพื่อจำแนกคุณลักษณะของการเปลี่ยนแปลงแนวคิด (Concept Drift) โดยอิงจากรูปแบบของการเปลี่ยนแปลงที่เกิดขึ้นในระบบ ซึ่งสามารถจำแนกได้ ดังนี้

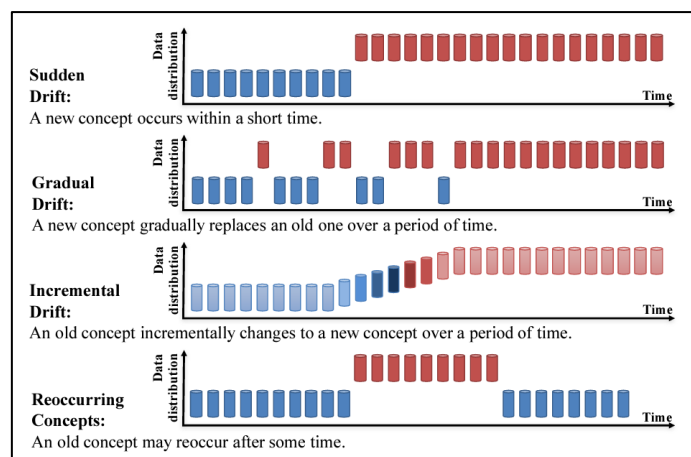
การเปลี่ยนแปลงแบบฉับพลัน (Sudden Drift) เกิดขึ้นเมื่อการแจกแจงของค่ากลุ่มเป้าหมาย (Target Distribution) เปลี่ยนแปลงจากแนวคิดหนึ่งไปสู่อีกแนวคิดหนึ่งอย่างรวดเร็วและเฉียบพลัน ช่วงเวลาหนึ่ง

การเปลี่ยนแปลงแบบค่อยเป็นค่อยไป (Gradual Drift) เกิดขึ้นเมื่อการแจกแจงของค่ากลุ่มเป้าหมายมีการเปลี่ยนแปลงอย่างต่อเนื่องจากแนวคิดหนึ่งไปสู่อีกแนวคิดหนึ่งแบบเป็นขั้นตอน

การเปลี่ยนแปลงแบบเพิ่มขึ้นทีละน้อย (Incremental Drift) เกิดขึ้นเมื่อแนวคิดใหม่เข้ามาแทนที่แนวคิดเดิมอย่างช้า ๆ และต่อเนื่องเป็นลำดับ โดยผู้วิจัยบางรายพิจารณาว่าการเปลี่ยนแปลงลักษณะนี้เป็นกลุ่มย่อยของการเปลี่ยนแปลงแบบค่อยเป็นค่อยไป (Krawczyk และคณะ, 2017) เนื่องจากทั้งสองกรณีมีลักษณะร่วมคือ แนวคิดใหม่เกิดขึ้นในระบบและเข้ามาแทนที่แนวคิดเดิมอย่างสมบูรณ์ อย่างไรก็ตาม ความแตกต่างคือ ในการเปลี่ยนแปลงแบบเพิ่มขึ้นทีละน้อย จะไม่มีเส้นแบ่งที่ชัดเจนระหว่างช่วงเวลาแนวคิดต่างกันปรากฏขึ้น (Zhong, Yang, Zhang, Li, และ Ren, 2021)

การเปลี่ยนแปลงแบบเกิดซ้ำ (Recurring Drift) เกิดขึ้นเมื่อแนวคิดที่เคยปรากฏในอดีตกลับมาเกิดขึ้นอีกครั้งหลังจากช่วงเวลาหนึ่ง ลักษณะของการเปลี่ยนแปลงประเภทนี้มีความคล้ายคลึงกับการเปลี่ยนแปลง

แบบค่อยเป็นค่อยไป เนื่องจากแนวคิดสองแนวมีการสลับกันปรากฏในระบบ อย่างไรก็ตาม ความแตกต่างหลักอยู่ที่ ช่วงการเปลี่ยนผ่าน (Transition Phase) โดยในกรณีของการเปลี่ยนแปลงแบบค่อยเป็นค่อยไป แนวคิดเดิมจะค่อย ๆ ลดลงและถูกแทนที่โดยแนวคิดใหม่อย่างต่อเนื่อง ในขณะที่การเปลี่ยนแปลงแบบเกิดซ้ำ แนวคิดเดิมจะกลับมาอีกครั้งหลังจากช่วงเวลาหนึ่ง (Ramírez-Gallego และคณะ, 2017)



ภาพที่ 3 การจำแนกประเภทของการเปลี่ยนแปลงแนวคิดตามลักษณะของรูปแบบการปรากฏ

ที่มา: <https://medium.com/quantumblack/guiding-principles-for-building-and-managing-effective-scalable-data-pipelines-for-machine-d73d052b1092> สืบค้นวันที่ 17 กรกฎาคม 2568

2.2 Cross-Validation ใน Time Series

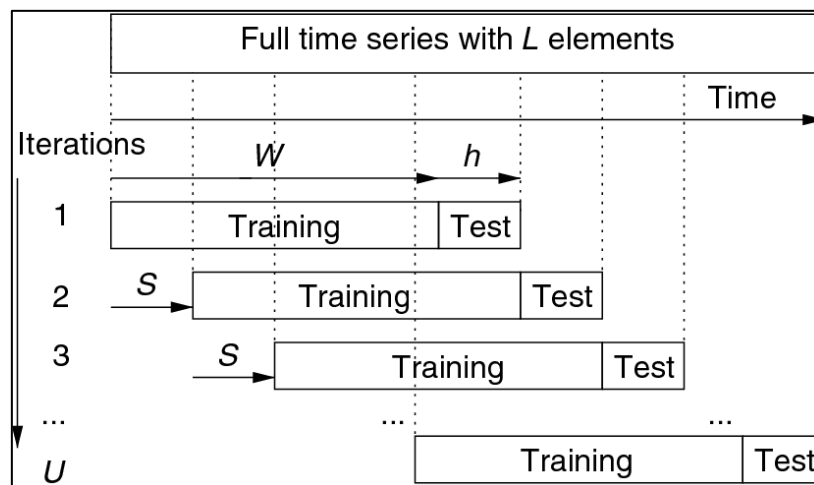
2.2.1 Rolling Window

เทคนิค Rolling Window ซึ่งมีความหมายตรงตัวคือ หน้าต่างเลื่อน ที่ใช้คำนวณค่าสถิติต่าง ๆ จากช่วงข้อมูลย่อยขนาดคงที่ โดยหน้าต่างนี้จะเลื่อนไปตามลำดับเวลาอย่างต่อเนื่อง โดยหลักการของ Rolling Window คือการใช้ช่วงข้อมูลจำนวนจำกัดที่กำหนดไว้ เช่น 7 วัน หรือ 30 จุดข้อมูลล่าสุด มาคำนวณค่าสถิติต่าง ๆ หรือใช้เป็นชุดข้อมูลฝึก (Training Set) ในการสร้างแบบจำลอง จากนั้นจะเลื่อนหน้าต่างนี้ไปข้างหน้าโดยทิ้งข้อมูลเก่าที่อยู่ด้านซ้ายออกและเพิ่มข้อมูลใหม่เข้ามาทางด้านขวา วิธีนี้ทำให้แบบจำลองหรือการคำนวณสถิติสามารถจับความเปลี่ยนแปลงล่าสุดในข้อมูลได้อย่างรวดเร็วและเหมาะสมกับสภาพแวดล้อมที่มีการเปลี่ยนแปลงอย่างต่อเนื่อง เช่น ตลาดหุ้นที่ผันผวน หรือการเปลี่ยนแปลงพฤติกรรมผู้บริโภค

ความสำคัญของ Rolling Window ยังอยู่ที่ความสามารถในการจัดการกับ Concept Drift หรือการเปลี่ยนแปลงของรูปแบบข้อมูลตามกาลเวลา ซึ่งเป็นปรากฏการณ์ที่พบบ่อยในข้อมูลจริง การเลือกใช้ Rolling Window โดยเฉพาะเมื่อหน้าต่างมีขนาดเหมาะสม จะช่วยลดอิทธิพลของข้อมูลเก่าที่อาจไม่เกี่ยวข้องหรือขัดแย้งกับสถานการณ์ปัจจุบัน ส่งผลให้แบบจำลองสามารถปรับตัวกับการเปลี่ยนแปลงนี้ได้ดีกว่าเทคนิคอื่น เช่น Expanding Window ที่เก็บข้อมูลเก่าทั้งหมดไว้

ในเชิงทฤษฎี Hyndman และ Athanasopoulos (2021) อธิบายว่า Rolling Window เป็นวิธีที่เหมาะสมในการทำ Cross-Validation สำหรับชุดข้อมูลอนุกรมเวลาที่มีการเปลี่ยนแปลงโครงสร้างอย่างต่อเนื่อง และเน้นการใช้ข้อมูลในช่วงเวลาล่าสุดเพื่อฝึกและทดสอบโมเดล ซึ่งช่วยหลีกเลี่ยงการใช้ข้อมูลอนาคตมาทำนายอดีตผิดลำดับ ทั้งนี้การเลือกขนาดของหน้าต่างใน Rolling Window ถือเป็นประเด็นสำคัญ เพราะขนาดหน้าต่างที่เล็กเกินไปอาจทำให้แบบจำลองมีความผันผวนสูงและเรียนรู้ไม่เพียงพอ ขณะที่หน้าต่างที่ใหญ่เกินไปอาจทำให้แบบจำลองตอบสนองต่อการเปลี่ยนแปลงได้ช้า และยังอาจสูญเสียประโยชน์ในการลดอิทธิพลของข้อมูลเก่า

ดังนั้น Rolling Window จึงเป็นเทคนิคที่ทรงพลังสำหรับการวิเคราะห์และสร้างแบบจำลองในอนุกรมเวลาที่มีการเปลี่ยนแปลง หรือในกรณีที่ความแม่นยำของการพยากรณ์ในระยะสั้นเป็นสิ่งจำเป็น เทคนิคนี้ช่วยให้แบบจำลองสามารถตอบสนองและปรับตัวได้อย่างเหมาะสมกับข้อมูลในโลกความเป็นจริงที่มีการเคลื่อนไหวและเปลี่ยนแปลงอยู่ตลอดเวลา



ภาพที่ 4 หลักการทำงานของ Rolling Window

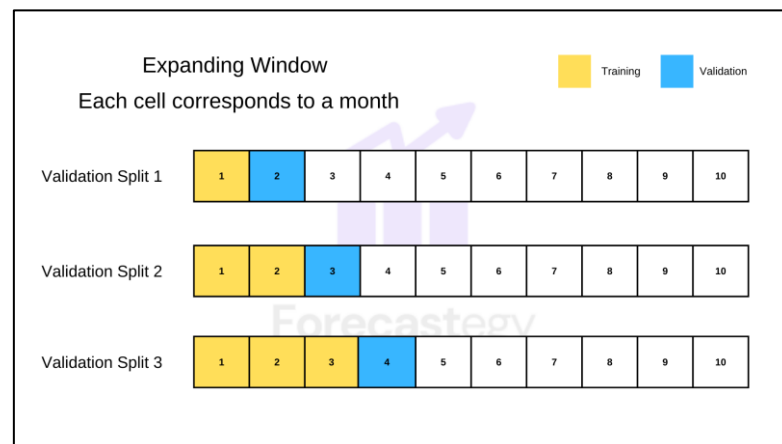
ที่มา: https://www.researchgate.net/figure/Schematic-of-the-rolling-window-procedure_fig6_334900650

สืบค้นวันที่ 17 กรกฎาคม 2568

2.2.2 Expanding Windows

ในการวิเคราะห์ข้อมูลแบบอนุกรมเวลา (Time Series Analysis) การแบ่งข้อมูลเพื่อประเมินประสิทธิภาพของแบบจำลองพยากรณ์ (Forecasting Model) มีความสำคัญอย่างยิ่งต่อการวัดความแม่นยำของแบบจำลองในสภาพแวดล้อมที่ใกล้เคียงกับการใช้งานจริง หนึ่งในเทคนิคที่ใช้บ่อยและเหมาะสมกับลักษณะข้อมูลตามเวลา คือเทคนิค Expanding Window Cross-Validation ซึ่งเป็นการประเมินแบบจำลองที่ใช้ข้อมูลจากอดีตเพิ่มขึ้นเรื่อย ๆ เพื่อเรียนรู้และทำนายอนาคต โดยสะท้อนหลักการเรียนรู้จาก ประสบการณ์สะสม ที่เพิ่มขึ้นตามลำดับเวลา

ตามแนวคิดของ Hyndman และ Athanasopoulos (2021) เทคนิค Expanding Window เป็นการจำลองกระบวนการที่ในแต่ละช่วงเวลาจะมีข้อมูลใหม่เข้ามาเติมในชุดข้อมูลฝึก (Training Set) โดยชุดข้อมูลฝึกจะขยายตัวมากขึ้นเรื่อย ๆ ในขณะที่ชุดข้อมูลทดสอบ (Test Set) ถูกเลื่อนตามลำดับเวลา วิธีนี้จึงมีความเหมาะสมกับงานพยากรณ์ที่ข้อมูลในอดีตยังคงมีประโยชน์ต่อการพยากรณ์อนาคต ทั้งนี้ลักษณะของการแบ่งข้อมูลในแต่ละรอบ (Fold) มักเป็นไปในรูปแบบ ดังภาพที่ 5 โดยการใช้ Expanding Window สามารถลดความลำเอียงของการประเมินโมเดลที่มักเกิดจากการใช้ข้อมูลในอนาคตมาทำนายอดีต ซึ่งผิดหลักของการวิเคราะห์ Time Series โดยสิ้นเชิง การแบ่งข้อมูลตามลำดับเวลานี้ จึงถือเป็นการเคารพโครงสร้างเชิงเวลาและเหมาะสมกับงานพยากรณ์จริง อย่างไรก็ตาม ในบางกรณีที่ข้อมูลอาจมีการเปลี่ยนแปลงของโครงสร้างหรือความสัมพันธ์ในข้อมูลตามกาลเวลา (เช่น การเกิด Concept Drift) การฝึกแบบจำลองด้วยข้อมูลทั้งหมดในอดีตอาจไม่เหมาะสม ด้วยเหตุนี้ Expanding Window จึงเป็นเครื่องมือที่มีประโยชน์อย่างมากในงานวิเคราะห์และพยากรณ์อนุกรมเวลา โดยเฉพาะในบริบทที่ต้องการประเมินแบบจำลองตามลำดับเวลาอย่างสมจริง และข้อมูลในอดีตยังคงมีคุณค่าต่อการพยากรณ์อนาคต



ภาพที่ 5 หลักการทำงานของ Expanding Window

ที่มา: <https://forecastegy.com/posts/time-series-cross-validation-python/> สืบค้นวันที่ 17 กรกฎาคม 2568

2.3 สถิติที่ใช้ตรวจ Drift

2.3.1 อัลกอริทึม Adaptive Windowing (ADWIN)

Bahar & Saad (2024) ได้กล่าวไว้ว่า ADWIN เป็นอัลกอริทึมการตรวจจับและประเมินการเปลี่ยนแปลงที่ช่วยแก้ปัญหาการเฝ้าติดตามค่าเฉลี่ยของชุดข้อมูลจำนวนเต็มแบบไบนารีหรือจำนวนจริง โดย ADWIN จะรักษาช่วงข้อมูล (Window) ที่มีความยาวผันแปรได้ เพื่อติดตามข้อมูลที่เพิ่งปรากฏไม่นาน ซึ่งความยาวสูงสุดของช่วงข้อมูลจะถูกกำหนดจากการสนับสนุนทางสถิติสำหรับข้อสันนิษฐานที่ว่า ค่าเฉลี่ยภายในช่วงข้อมูลนั้นคงที่อยู่เสมอ

แนวทางของ ADWIN ตั้งข้อสมมติฐานว่า เมื่อมีช่วงข้อมูลย่อยสองช่วงของ W ซึ่งพิจารณาว่ามีขนาดใหญ่มากพอที่จะแสดงค่าเฉลี่ยที่ถือว่าแตกต่างกันพอสมควรแล้ว จะสามารถอนุมานได้ว่าค่าที่คาดการณ์ไว้ที่เกี่ยวข้องนั้นแตกต่างกัน และส่งผลให้ส่วนที่เก่ากว่าของช่วงข้อมูลจะถูกกำจัดออกไป การใช้ Hoeffding bound ทำให้สามารถให้คำนิยามที่เฉพาะเจาะจงมากขึ้นสำหรับคำว่า ใหญ่พอ และแตกต่างกันพอ ได้ ในการวิเคราะห์ขั้นสุดท้าย การทดสอบจะประเมินว่าค่าเฉลี่ยของช่วงข้อมูลย่อยสองช่วงนั้นเกินค่า ϵ_{cut} หรือไม่

$$m := \frac{2}{\frac{1}{|W_0|} + \frac{1}{|W_1|}}$$

$$\epsilon_{cut} := \sqrt{\frac{1}{2m} \cdot \ln \frac{|W|}{\delta}}$$

m : ค่าเฉลี่ยฮาร์มอนิก (Harmonic mean) ของ $|W_0|$ และ $|W_1|$

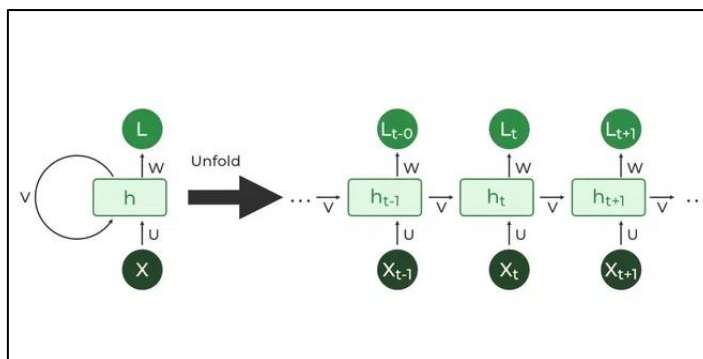
2.4 แบบจำลองพยากรณ์

2.4.1 Recurrent Neural Network (RNN)

Recurrent Neural Network (RNN) เป็นโครงข่ายประสาทเทียมที่ออกแบบมาเพื่อประมวลผลข้อมูลที่มีลักษณะเป็นลำดับ (Sequential Data) เช่น ข้อความ เสียง หรือชุดข้อมูลเวลา ด้วยความสามารถในการเก็บข้อมูลจากช่วงเวลาก่อนหน้า RNN จึงสามารถเรียนรู้บริบทและความสัมพันธ์ภายในข้อมูลตามลำดับได้อย่างมีประสิทธิภาพ อย่างไรก็ตาม RNN แบบดั้งเดิมยังมีข้อจำกัดในเรื่องการเรียนรู้ข้อมูลระยะยาว

Goodfellow, Bengio และ Courville (2016) อธิบายว่า โครงสร้างและหลักการทำงานของ RNN มีลักษณะเป็นโครงข่ายประสาทเทียมที่ประกอบด้วยหน่วยซ้ำ ๆ ที่เชื่อมโยงกันแบบวนลูป ทำให้แต่ละหน่วยไม่เพียงรับข้อมูลจากอินพุตในช่วงเวลาปัจจุบันเท่านั้น แต่ยังรับข้อมูลสถานะ (Hidden State) จากช่วงเวลาก่อนหน้าอีกด้วย ด้วยเหตุนี้ RNN จึงสามารถจดจำและใช้บริบทของข้อมูลก่อนหน้ามาประมวลผลร่วมกับข้อมูลปัจจุบันได้ ซึ่งเป็นคุณสมบัติที่จำเป็นสำหรับการประมวลผลข้อมูลตามลำดับ เช่น การทำนายคำถัดไปในประโยค หรือการวิเคราะห์ข้อมูลเวลา ดังภาพที่ 6

Hochreiter และ Schmidhuber (1997) ชี้ให้เห็นว่า RNN แบบดั้งเดิมมีปัญหาที่เรียกว่า Vanishing Gradient ซึ่งหมายถึงปัญหาที่เกิดขึ้นในกระบวนการเรียนรู้ด้วยวิธี Backpropagation เมื่อข้อมูลมีลำดับยาวมาก Gradients ที่ใช้ปรับน้ำหนักในโมเดลจะค่อยๆ ลดค่าลงจนแทบไม่ส่งผล ทำให้โมเดลไม่สามารถเรียนรู้ความสัมพันธ์ระยะยาวได้อย่างมีประสิทธิภาพ นั่นหมายความว่า RNN อาจลืมข้อมูลที่สำคัญในช่วงเวลาที่ห่างไกลออกไป ส่งผลให้ประสิทธิภาพในการวิเคราะห์ข้อมูลลำดับยาวลดลง



ภาพที่ 6 โครงสร้างและหลักการทำงานของ RNN

ที่มา: <https://www.geeksforgeeks.org/machine-learning/introduction-to-recurrent-neural-network/>

สืบค้นวันที่ 17 กรกฎาคม 2568

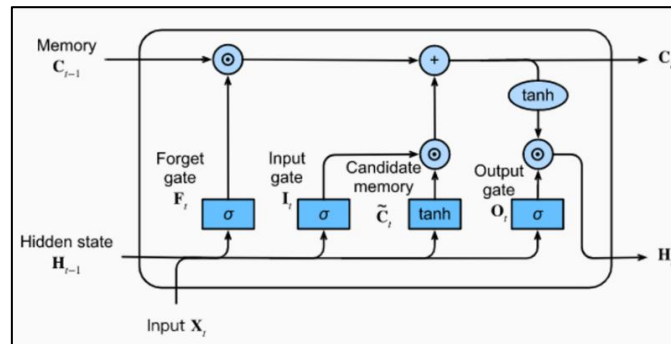
2.4.2 Long Short-Term Memory (LSTM)

Long Short-Term Memory (LSTM) เป็นโครงข่ายประสาทเทียมเชิงลำดับ (Sequential Neural Network) ที่ได้รับการพัฒนาต่อยอดมาจากโครงข่าย Recurrent Neural Network (RNN) เพื่อแก้ไขข้อจำกัดของ RNN ในการเรียนรู้ข้อมูลย้อนหลังในระยะยาว โดย RNN มีหลักการทำงานที่อาศัยผลลัพธ์จากโหนดก่อนหน้า (Previous Output) มาใช้เป็นข้อมูลนำเข้า (Input) ของโหนดถัดไปในลำดับเวลา ซึ่งส่งผลให้โครงข่ายสามารถเรียนรู้รูปแบบของข้อมูลที่มีลักษณะต่อเนื่อง (Greff, Srivastava และ คณะ, 2017)

อย่างไรก็ตาม แม้ RNN จะสามารถนำข้อมูลจากอดีตมาใช้ในการคาดการณ์อนาคตได้ แต่ก็มีข้อจำกัดสำคัญ คือ ความสามารถในการเรียนรู้ข้อมูลย้อนหลังมักถูกจำกัดอยู่เพียงระยะสั้น เนื่องจากปัญหาการลดทอนของค่า Gradient ในระหว่างกระบวนการ Backpropagation Through Time (BPTT) ซึ่งเป็นที่รู้จักในชื่อของ Vanishing Gradient Problem ปัญหาดังกล่าวส่งผลให้ RNN ไม่สามารถเรียนรู้ความสัมพันธ์ในลำดับข้อมูลที่มีความยาวมากได้อย่างมีประสิทธิภาพ เพื่อแก้ไขข้อจำกัดข้างต้น Hochreiter และ Schmidhuber (1997) จึงได้เสนอ Long Short-Term Memory (LSTM) ซึ่งเป็นโครงข่าย RNN รูปแบบหนึ่ง ที่ออกแบบให้สามารถเก็บรักษาข้อมูลย้อนหลังในระยะยาวได้อย่างมีประสิทธิภาพ โดย LSTM มีการเพิ่มกลไกพิเศษในแต่ละโหนด เรียกว่า ประตู (Gate) ซึ่งมีหน้าที่ในการควบคุมการไหลของข้อมูลภายในโครงข่าย ดังภาพที่ 7 ได้แก่

- Forget Gate ทำหน้าที่ตัดข้อมูลที่ไม่น่าจำเป็นออก
- Input Gate ควบคุมข้อมูลใหม่ที่จะถูกเพิ่มเข้าไปในสถานะของหน่วยความจำ
- Output Gate คัดกรองข้อมูลที่จำเป็นเพื่อส่งออกเป็นผลลัพธ์ในแต่ละช่วงเวลา

ด้วยโครงสร้างข้างต้น LSTM จึงสามารถจัดการกับข้อมูลที่มีลักษณะลำดับต่อเนื่องในระยะยาวได้ดีกว่า RNN ทั่วไป และได้รับความนิยมอย่างแพร่หลายในงานวิเคราะห์ข้อมูลอนุกรมเวลาและการพยากรณ์ข้อมูล (Jozefowicz, Zaremba, และ Sutskever, 2015)



ภาพที่ 7 โครงสร้างและหลักการทำงานของ LSTM

ที่มา: <https://medium.com/@ottaviocalzone/an-intuitive-explanation-of-lstm-a035eb6ab42c>

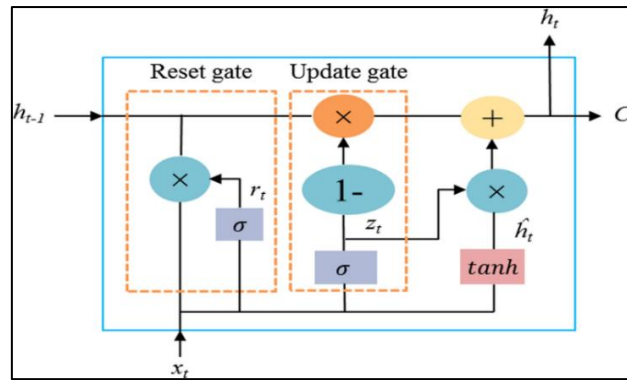
สืบค้นวันที่ 17 กรกฎาคม 2568

2.4.3 Gated Recurrent Unit (GRU)

GRU หรือ Gated Recurrent Unit เป็นโครงข่ายประสาทเทียมชนิดหนึ่งที่พัฒนาต่อยอดมาจาก Recurrent Neural Network (RNN) และ Long Short-Term Memory (LSTM) เพื่อแก้ปัญหา Vanishing Gradient ที่เกิดขึ้นใน RNN โดย GRU ถูกออกแบบมาให้มีความเรียบง่ายและมีประสิทธิภาพสูงขึ้นในการจดจำข้อมูลระยะยาว GRU ใช้กลไก Gates เพื่อตัดสินใจว่าจะเก็บข้อมูลจากอดีตมากน้อยแค่ไหน

โดยมี Gate หลัก 2 ตัว ดังภาพที่ 8 คือ Update Gate ซึ่งควบคุมการอัปเดตสถานะของหน่วยความจำ และ Reset Gate ซึ่งควบคุมว่าจะลืมข้อมูลก่อนหน้าเท่าใด กลไกนี้ช่วยให้ GRU สามารถจัดการข้อมูลลำดับยาวได้ดีขึ้น โดยมีโครงสร้างที่ง่ายกว่าหรือเบากว่า LSTM แต่ยังคงประสิทธิภาพในการจดจำข้อมูลระยะยาวไว้ได้

ข้อดีของ GRU คือมีจำนวนพารามิเตอร์น้อยกว่า LSTM ทำให้ฝึกอบรมได้เร็วกว่าและเหมาะสำหรับงานที่มีข้อจำกัดด้านข้อมูลหรือเวลาคำนวณ ในหลายกรณี GRU ให้ผลลัพธ์ใกล้เคียงกับ LSTM จึงได้รับความนิยมอย่างแพร่หลายในการประมวลผลข้อมูลตามลำดับ เช่น การประมวลผลภาษาธรรมชาติ การรู้จำเสียง และการพยากรณ์ข้อมูลชุดเวลา (Chung, Gulcehre, Cho และ Bengio, 2014)



ภาพที่ 8 โครงสร้างและหลักการทำงานของ GRU

ที่มา: <https://blog.gopenai.com/comparing-rnn-lstm-gru-and-transformer-12dd8c134255>

สืบค้นวันที่ 17 กรกฎาคม 2568

2.4.4 Linear Regression

การถดถอยเชิงเส้น (Linear Regression) เป็นเทคนิคทางสถิติที่ใช้กันอย่างแพร่หลายในการวิเคราะห์ความสัมพันธ์ระหว่างตัวแปร โดยมีวัตถุประสงค์เพื่อศึกษาความสัมพันธ์ระหว่างตัวแปรอิสระ (Independent Variable) และตัวแปรตาม (Dependent Variable) โดยสมมุติว่าความสัมพันธ์ระหว่างตัวแปรทั้งสองมีลักษณะเชิงเส้น (James, Witten, Hastie, และ Tibshirani, 2021)

$$Y = \beta_0 + \beta_1 X + \epsilon$$

โดยที่

Y คือค่าของตัวแปรตาม

X คือค่าของตัวแปรอิสระ

β_0 คือค่าคงที่ (intercept)

β_1 คือค่าสัมประสิทธิ์ของตัวแปรอิสระ (slope)

ϵ คือค่าคลาดเคลื่อน (error term) ของแบบจำลอง

2.5 Metric ในการประเมิน

Yuk และThongkam (2018) ได้อธิบายค่าเฉลี่ยความคลาดเคลื่อนสัมบูรณ์ (Mean Absolute Error: MAE) และ ค่ารากที่สองของความคลาดเคลื่อนเฉลี่ยกำลังสอง (Root Mean Square Error: RMSE) ไว้ดังนี้

2.5.1 ค่าเฉลี่ยความคลาดเคลื่อนสัมบูรณ์ (Mean Absolute Error: MAE)

คือ ค่าความแตกต่างโดยเฉลี่ยระหว่างค่าจากการพยากรณ์ และค่าจริง ดังสมการต่อไปนี้

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

เมื่อ y_i คือ ค่าจากการพยากรณ์

$\hat{y}_i$ คือ ค่าจริง

n จำนวนข้อมูลทั้งหมด

2.5.2 ค่ารากที่สองของความคลาดเคลื่อนเฉลี่ยกำลังสอง (Root Mean Square Error: RMSE)

คือ การวัดความคลาดเคลื่อนเพื่อเปรียบเทียบความแตกต่างระหว่างค่าจากการพยากรณ์และค่าจริงเฉลี่ยกำลังสอง ดังสมการต่อไปนี้

$$RMSE = \sqrt{\frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{n}}$$

จากค่าที่ได้ ถ้าค่า MAE และ RMSE ต่ำแสดงว่าค่าพยากรณ์มีความใกล้เคียงกับค่าจริง ซึ่งหมายถึงแบบจำลอง มีประสิทธิภาพในการพยากรณ์สูง

2.6 งานวิจัยที่เกี่ยวข้อง

Zhao และ Shen (2025) ได้เสนอกรอบแนวคิดใหม่ในการปรับตัวของโมเดลสำหรับการพยากรณ์อนุกรมเวลาในสถานการณ์ที่เกิด Concept Drift อย่างต่อเนื่อง ซึ่งเป็นลักษณะทั่วไปของข้อมูลจริงในรูปแบบ Online Time Series โดยงานวิจัยนี้เน้นประเด็นสำคัญคือ ความล่าช้าในการรับรู้ค่าจริง (Ground Truth Delay) หลังการพยากรณ์ ซึ่งทำให้เกิดช่องว่างของเวลา (Temporal Gap) ระหว่างข้อมูลฝึก (Training Sample) และข้อมูลทดสอบ (Test Sample) ส่งผลให้โมเดลปรับตัวตามข้อมูลที่ล้าสมัย และลดประสิทธิภาพในการพยากรณ์ เพื่อแก้ปัญหาดังกล่าว ผู้วิจัยได้พัฒนาแนวทางชื่อว่า Proceed (Proactive Model Adaptation) ซึ่งมีจุดเด่นคือการปรับตัวล่วงหน้า (Proactive Adaptation) โดยใช้การประเมินระดับของ Concept Drift ระหว่างข้อมูลฝึกกับข้อมูลทดสอบ แล้วนำมาใช้ในการสร้างการปรับค่าพารามิเตอร์ของโมเดลที่เหมาะสมกับตัวอย่างใหม่ วิธีการนี้ไม่เพียงใช้ Gradient Descent จาก Feedback ที่ล่าช้าเท่านั้น แต่ยังแยกการประมาณการ Drift และการปรับโมเดลออกจากกันอย่างชัดเจน Proceed ประกอบด้วยสองโมดูลสำคัญ ได้แก่ Concept Encoder ทำหน้าที่เข้ารหัสตัวแทนของแนวคิด

(Concept Representation) จากข้อมูลฝึกและข้อมูลทดสอบ Adaptation Generator สร้างการปรับค่าพารามิเตอร์โดยใช้โครงสร้างแบบ Bottleneck เพื่อหลีกเลี่ยงการใช้พารามิเตอร์จำนวนมากที่อาจนำไปสู่การ Overfitting นอกจากนี้ งานวิจัยยังเสนอการฝึกฝนแบบ Offline ด้วยข้อมูล Drift จำลองที่สร้างขึ้นจากการสับลำดับของข้อมูลในอดีต เพื่อเรียนรู้ความสัมพันธ์ระหว่างรูปแบบ Drift กับการเปลี่ยนแปลงของพารามิเตอร์ ซึ่งช่วยให้โมเดลสามารถ Generalize ได้ดีในสถานการณ์จริงที่ Concept Drift มีความหลากหลาย ผลการทดลองในชุดข้อมูลจริง 5 ชุด ได้แก่ ETTh2, ETTm1, Weather, ECL และ Traffic เปรียบเทียบกับวิธีมาตรฐาน เช่น FSNet, OneNet และ SOLID++ พบว่า Proceed สามารถลดค่าเฉลี่ยของ MSE ได้มากกว่า 21.9% และมีความเร็วในการปรับตัวที่สูงกว่า โดยใช้หน่วยความจำน้อยกว่าวิธี Ensemble หรือ Fine-Tune แบบเดิมอย่างมีนัยสำคัญ

Chen และคณะ (2024) ได้เสนอแนวทางใหม่ในการจัดการกับปัญหา Context-Driven Distribution Shift (CDS) ซึ่งเป็นความเปลี่ยนแปลงของการแจกแจงข้อมูลที่เกิดจากบริบททางเวลา เช่น ช่วงเวลา (Temporal Segments), วัฏจักรฤดูกาล (Periodic Phases) หรือแม้กระทั่งเหตุการณ์ที่ไม่ได้สังเกตโดยตรง (Unobserved Contexts) ปัญหานี้ส่งผลให้โมเดลพยากรณ์อนุกรมเวลาแบบดั้งเดิม เช่น Autoformer, Informer หรือ PatchTST เกิดความเอนเอียงในการพยากรณ์ (Prediction Bias) และไม่สามารถตอบสนองต่อข้อมูลที่เปลี่ยนแปลงได้อย่างแม่นยำ จากการทดลองกับชุดข้อมูลจริง 8 ชุด รวมถึง Electricity, Traffic, Illness และชุดข้อมูล ETT ต่าง ๆ พบว่า SOLID ช่วยเพิ่มประสิทธิภาพของโมเดลได้อย่างมีนัยสำคัญ โดยเฉพาะในกรณีที่มี CDS สูง (เช่น $\log_{10} \delta > -3.2$) ซึ่งสามารถเพิ่มความแม่นยำในการพยากรณ์ได้ถึง 15.1% และยังมีผลดีแม้ในกรณี CDS ต่ำ นอกจากนี้ Reconditionor ยังสามารถใช้เป็นตัวบ่งชี้ว่าควรปรับโมเดลหรือไม่ โดยให้ค่าความแม่นยำของการตรวจจับสูงถึง 89.3%

Liu และคณะ (2023) ได้ศึกษาแนวทางการจัดการกับ Concept Drift หรือการเปลี่ยนแปลงของรูปแบบข้อมูลในงานพยากรณ์อนุกรมเวลา โดยเน้นเฉพาะในบริบทของ Global Forecasting Models (GFMs) ซึ่งเป็นแบบจำลองที่เรียนรู้จากหลายชุดข้อมูลอนุกรมเวลาในเวลาเดียวกัน งานวิจัยนี้ชี้ให้เห็นว่าหลายเทคนิคที่มีอยู่เดิมมุ่งเน้นการจัดการกับ Concept Drift ในปัญหาการจำแนกประเภท (Classification) และยังไม่เพียงพอสำหรับงานพยากรณ์ที่ข้อมูลมีการเปลี่ยนแปลงอย่างต่อเนื่อง เพื่อตอบโจทยดังกล่าว ผู้วิจัยได้เสนอวิธีการใหม่สองวิธี คือ Error Contribution Weighting (ECW) และ Gradient Descent Weighting (GDW) ซึ่งเป็นการผสมผลลัพธ์ของแบบจำลองย่อยสองตัว ได้แก่ แบบจำลองที่ฝึกจากข้อมูลล่าสุดเท่านั้น และแบบจำลองที่ฝึกจากข้อมูลทั้งหมด แนวคิดหลักคือการให้น้ำหนักกับแบบจำลองแต่ละตัวโดยอิงจากความผิดพลาดในการทำนายที่ผ่านมา (Prediction Error) ซึ่งน้ำหนักนี้จะถูกปรับแบบต่อเนื่องตามลักษณะของข้อมูลที่เปลี่ยนไป การทดลองที่ใช้ชุดข้อมูลจำลองที่มีลักษณะของ Concept Drift แบบต่าง ๆ ได้แก่ แบบฉับพลัน (Sudden), แบบเพิ่มขึ้นทีละน้อย (Incremental) และแบบค่อยเป็นค่อยไป (Gradual) แสดงให้เห็นว่า GDW และ ECW มีประสิทธิภาพเหนือกว่าแบบจำลองทั่วไป เช่น ARIMA, ETS และ LightGBM ทั้งในแง่ของความแม่นยำ (MAE, RMSE) และความสามารถในการตอบสนองต่อ Drift ได้อย่างทันท่วงที โดยเฉพาะ

GDW ซึ่งใช้เทคนิค Gradient Descent ในการปรับน้ำหนักแบบจำลองย่อย แสดงผลลัพธ์ที่ดีที่สุดในเกือบทุกสถานการณ์และผ่านการทดสอบทางสถิติอย่างเข้มงวด งานวิจัยนี้จึงเป็นการปูทางสำคัญสำหรับการพัฒนาเทคนิคจัดการกับ Concept Drift ในระบบพยากรณ์แบบหลายชุดข้อมูล ซึ่งกำลังเป็นที่นิยมในยุคของ Big Data และ Machine Learning

Gama และคณะ (2013) ได้นำเสนอการสำรวจงานวิจัยด้านการเรียนรู้ที่สามารถปรับตัวกับการเปลี่ยนแปลงของข้อมูล หรือที่เรียกว่า Concept Drift ซึ่งเป็นลักษณะเฉพาะของปัญหาในสภาพแวดล้อมที่ข้อมูลมีการเปลี่ยนแปลงเมื่อเวลาผ่านไป โดยเฉพาะในการเรียนรู้แบบออนไลน์ ที่โมเดลต้องสามารถปรับตัวได้ต่อการเปลี่ยนแปลงของความสัมพันธ์ระหว่างตัวแปรต้น (Input) และตัวแปรเป้าหมาย (Target) อย่างต่อเนื่อง งานวิจัยนี้ได้จัดระบบและจำแนกแนวทางการจัดการกับ Concept Drift ออกเป็น 4 โมดูลหลัก ได้แก่ Memory คือการจัดการกับข้อมูลใหม่และกลไกการลืม (Forgetting mechanisms) เช่น Sliding Window, Sampling, และการถ่วงน้ำหนักข้อมูลเก่า Change Detection คือวิธีการตรวจจับการเปลี่ยนแปลง เช่น Sequential Analysis, Statistical Process Control (SPC), และการเปรียบเทียบการแจกแจงของข้อมูลสองช่วงเวลา (เช่น ADWIN) Learning คือวิธีการเรียนรู้ เช่น การเรียนแบบ Incremental, Online, Retraining และแนวทางการจัดการแบบจำลองทั้งแบบเดี่ยวและแบบ Ensemble และ Loss Estimation คือการประเมินประสิทธิภาพของโมเดลอย่างต่อเนื่อง เพื่อตัดสินใจว่าควรปรับหรือเปลี่ยนแบบจำลองหรือไม่ บทความยังได้จำแนกประเภทของ Drift ออกเป็น Real Concept Drift ($p(y|X)$ เปลี่ยน) และ Virtual Drift ($p(X)$ เปลี่ยนโดยไม่กระทบ $p(y|X)$) พร้อมยกตัวอย่างการเปลี่ยนแปลงในรูปแบบต่าง ๆ เช่น การเปลี่ยนแปลงแบบฉับพลัน (Sudden), แบบค่อยเป็นค่อยไป (Gradual), และการเกิดซ้ำของแนวคิดเดิม (Recurring Concept) รวมถึงการนำไปประยุกต์ใช้งานในหลายสาขา เช่น ระบบควบคุมโรงงานอุตสาหกรรม ระบบพยากรณ์พลังงานลม การกรองสแปมและการจำแนกข้อความในอีเมล ระบบแนะนำ (Recommender Systems) ที่ผู้ใช้มีพฤติกรรมเปลี่ยนแปลงตามกาลเวลา และการเรียนรู้ในยานพาหนะไร้คนขับ (Autonomous Vehicles) สิ่งที่ทำให้งานวิจัยนี้โดดเด่นคือการรวบรวมและสังเคราะห์องค์ความรู้จากหลายสาขา ทั้งด้าน Machine Learning, Data Mining และ Stream Processing เข้าด้วยกัน พร้อมทั้งเสนอ Taxonomies หรือแผนผังการจำแนกประเภทของกลยุทธ์และอัลกอริธึมที่สามารถนำไปประยุกต์ใช้ร่วมกันแบบโมดูลาร์ (Modular Combination) ได้ งานวิจัยนี้จึงเป็นกรอบพื้นฐานที่สำคัญสำหรับโครงการวิจัยเกี่ยวกับการพยากรณ์ข้อมูลที่มี Concept Drift และเป็นแหล่งอ้างอิงหลักที่ช่วยให้เข้าใจการออกแบบระบบเรียนรู้ที่สามารถปรับตัวได้ในสถานะที่ไม่แน่นอนและเปลี่ยนแปลงอยู่เสมอ

Bifet และ Gavalda (2007) ได้นำเสนออัลกอริธึมที่ชื่อว่า ADWIN (Adaptive Windowing) ซึ่งเป็นแนวทางใหม่ในการจัดการกับปัญหาการเปลี่ยนแปลงของข้อมูลตามเวลา (Time-Changing Data) หรือปัญหา Concept Drift ในกระบวนการเรียนรู้จากข้อมูลแบบต่อเนื่อง (Data Streams) โดย ADWIN ใช้แนวคิดของ หน้าต่างเลื่อนแบบปรับขนาดได้ (Adaptive Sliding Window) ที่สามารถเปลี่ยนขนาดของหน้าต่างโดยอัตโนมัติตามอัตราการเปลี่ยนแปลงของข้อมูล กล่าวคือ เมื่อข้อมูลมีแนวโน้มคงที่ ADWIN

จะขยายหน้าต่างให้ใหญ่ขึ้นเพื่อเก็บข้อมูลมากขึ้น แต่เมื่อมีการเปลี่ยนแปลงที่สำคัญเกิดขึ้นในข้อมูล ระบบจะลดขนาดของหน้าต่างลงเพื่อลบข้อมูลเก่าที่อาจไม่สอดคล้องกับสถานการณ์ปัจจุบันออกไป จุดเด่นของ ADWIN อยู่ที่ความสามารถในการให้ ขอบเขตทางทฤษฎีที่ชัดเจน เกี่ยวกับอัตราความผิดพลาด ทั้งในกรณี False Positive และ False Negative โดยอิงจาก Hoeffding Bound และสามารถนำไปใช้ ร่วมกับตัวจำแนกประเภท เช่น Naïve Bayes ได้ทั้งในรูปแบบการตรวจสอบความคลาดเคลื่อนของโมเดล และในรูปแบบที่เป็นส่วนหนึ่งของกลไกการประมาณค่าความน่าจะเป็นแบบมีเงื่อนไขที่อัปเดตตามข้อมูลล่าสุด ต่อมา ผู้วิจัยได้พัฒนาอัลกอริธึมเวอร์ชันที่มีประสิทธิภาพยิ่งขึ้นในชื่อว่า ADWIN2 ซึ่งลดความซับซ้อนด้านเวลา และหน่วยความจำ ด้วยการนำแนวคิดจากโครงสร้างข้อมูลแบบ Exponential Histogram มาใช้ ทำให้สามารถประมวลผลข้อมูลใหม่ได้อย่างรวดเร็วและใช้หน่วยความจำอย่างมีประสิทธิภาพ โดยยังคงประสิทธิภาพในการตรวจจับการเปลี่ยนแปลงไว้ได้เช่นเดิม จากผลการทดลองทั้งในชุดข้อมูล สังเคราะห์และชุดข้อมูลจริง เช่น Electricity Market Dataset แสดงให้เห็นว่า ADWIN2 สามารถปรับตัวได้ดี กับอัตราการเปลี่ยนแปลงที่หลากหลายของข้อมูล และให้ผลลัพธ์ที่มีความแม่นยำสูงเมื่อเปรียบเทียบกับวิธี ที่ใช้หน้าต่างคงที่ โดยเฉพาะในสถานการณ์ที่การเปลี่ยนแปลงเกิดขึ้นในช่วงเวลาที่ไม่แน่นอน งานวิจัยนี้ จึงมีความเกี่ยวข้องอย่างยิ่งกับการพัฒนาโมเดลที่สามารถเรียนรู้จากข้อมูลที่เปลี่ยนแปลงตามเวลาได้อย่างมีประสิทธิภาพ โดยเฉพาะในบริบทของการพยากรณ์ การตรวจจับ Drift และการบริหารข้อมูลในระบบ สตรีมมิงอย่างยั่งยืน

บทที่ 3

วิธีดำเนินการวิจัย

ในการวิจัยครั้งนี้ ผู้วิจัยได้ดำเนินการตามขั้นตอน ดังนี้

1. การเก็บรวบรวมข้อมูล
2. การเตรียมข้อมูลและการสร้างตัวแปร (Feature Engineering)
3. การกำหนดตัวแปรคุณลักษณะ (X) และตัวแปรเป้าหมาย (y)
4. Adaptive Drift Detection Process Flowchart

3.1 การเก็บรวบรวมข้อมูล

ในการศึกษาครั้งนี้ ผู้วิจัยใช้ข้อมูลราคาหุ้นในตลาดหลักทรัพย์เป็นชุดข้อมูลหลัก โดยเลือกใช้ราคาหุ้นของ บริษัท Meta Platforms, Inc. ซึ่งเป็นบริษัทในกลุ่มอุตสาหกรรม Interactive Media & Services / Social Media บริษัท NVIDIA Corporation ซึ่งเป็นบริษัทในกลุ่มอุตสาหกรรม Semiconductors และ Tesla, Inc. ซึ่งเป็นบริษัทในกลุ่มอุตสาหกรรม Automobile Manufacturers (EVs) โดยข้อมูลดังกล่าวได้ถูกรวบรวมจากแหล่งข้อมูลออนไลน์ที่เชื่อถือได้ เช่น Yahoo Finance ผ่านการเรียกใช้งานด้วยโปรแกรมภาษา Python ผ่านไลบรารี yfinance ช่วงเวลาของข้อมูลที่นำมาใช้ในการศึกษา โดยเก็บข้อมูลช่วงระยะเวลา 10 ปี ตั้งแต่วันที่ 2 มกราคม พ.ศ. 2558 ถึง 31 ธันวาคม พ.ศ. 2567 โดยเป็นข้อมูลรายวัน (Daily Frequency) ที่ประกอบด้วยตัวแปรสำคัญ ได้แก่ วันที่ (Date), ราคาปิด (Close), ราคาสูงสุด (High), ราคาต่ำสุด (Low) และปริมาณการซื้อขาย (Volume) โดยข้อมูลถูกจัดเก็บในรูปแบบไฟล์ CSV และมีขั้นตอนการจัดกาเบื้องต้น คือ แปลง Date เป็นชนิด Datetime เรียงลำดับข้อมูลตามเวลา (Sort_values) และกรองข้อมูลให้มีเฉพาะวันทำการที่สมบูรณ์

3.2 การเตรียมข้อมูลและการสร้างตัวแปร (Feature Engineering)

3.2.1. การนำเข้าข้อมูล

ขั้นตอนการดำเนินงานจะเริ่มต้นจากการนำชุดข้อมูลเพื่อมาทำการแสดงรายละเอียด โดยยังไม่มี การสร้างฟีเจอร์เพิ่มเติม

3.2.2. ทำการเช็คข้อมูลมีค่าว่าง (Missing Data) และตรวจสอบจำนวนแถวข้อมูลที่ซ้ำกัน

ถือว่าเป็นขั้นตอนสำคัญในกระบวนการวิเคราะห์ข้อมูล เนื่องจากค่าว่างและแถวข้อมูลที่ซ้ำกันสามารถส่งผลกระทบต่อความถูกต้องและความน่าเชื่อถือของการวิเคราะห์หรือการสร้างโมเดลได้

3.2.3 การวิเคราะห์ข้อมูล

จะทำการวิเคราะห์ข้อมูลต่างๆ เพื่อค้นหาข้อมูลที่สำคัญที่แฝงอยู่ในกลุ่มข้อมูลนั้น โดยจะทำการหารูปแบบความเชื่อมโยงระหว่างกัน เริ่มจากการหาค่าทางสถิติเบื้องต้น

3.2.4 เตรียมข้อมูลสำหรับการสร้างแบบจำลองการพยากรณ์

ได้มีการประมวลผลข้อมูลเบื้องต้นด้วยเทคนิคการสร้างตัวแปรใหม่ (Feature Engineering) เพื่อใช้เป็นตัวแปรต้นในโมเดล ซึ่งตัวแปรที่ได้ประกอบด้วย

Return คือ ค่าผลตอบแทนรายวัน (Daily Return) ที่ได้จากการคำนวณอัตราการเปลี่ยนแปลงของราคาปิดในแต่ละวัน โดยใช้สูตร
$$\text{Return}_t = \frac{\text{Close}_t - \text{Close}_{t-1}}{\text{Close}_{t-1}}$$

Volatility คือ ความผันผวนของราคาปิดภายในช่วงเวลา 7 วัน โดยคำนวณจากส่วนเบี่ยงเบนมาตรฐานแบบเลื่อนช่วง (Rolling Standard Deviation) ซึ่งช่วยให้สามารถตรวจสอบระดับความเสี่ยงของราคาในช่วงเวลานั้นได้อย่างมีประสิทธิภาพ โดยใช้สูตร

$$\sigma_t = \sqrt{\frac{1}{N} \sum_{i=1}^7 (P_{t-i} - \bar{P})^2}$$

Volume_Log คือ ค่าล็อกของปริมาณการซื้อขายในแต่ละวัน เพื่อปรับให้ข้อมูลปริมาณการซื้อขายที่มีการกระจุกตัวสูง (Skewed) มีการกระจายตัวที่เหมาะสมมากยิ่งขึ้น และสามารถนำไปใช้ในโมเดลได้อย่างมีประสิทธิภาพ โดยใช้สูตร

$$\text{Volume_Log} = \log(1 + \text{Volume})$$

Return_Volume คือ ผลคูณของ Return และ Volume_Log เพื่อดูว่าการเปลี่ยนแปลงราคามีความสัมพันธ์กับปริมาณการซื้อขายมากน้อยแค่ไหน

$$\text{Return_Volume}_t = \text{Return}_t \times \text{Volume_Log}_t$$

3.3 การกำหนดตัวแปรคุณลักษณะ (X) และตัวแปรเป้าหมาย (y)

- ตัวแปร X (Features) ประกอบด้วยตัวแปรฟีเจอร์ที่สร้างขึ้น ได้แก่
 - Return , Volatility , Volume_Log , Return_Volume
- ตัวแปร y (Target) คือราคาปิดของหุ้นในวันนั้น ๆ (Close) ซึ่งเป็นค่าที่ต้องการให้โมเดลทำนาย

3.4 การตรวจจับ Concept Drift (Drift Detection)

3.4.1 หลักการของ ADWIN

- ใช้ Adaptive Windowing (ADWIN) เพื่อตรวจจับการเปลี่ยนแปลงการกระจายของข้อมูล
- พารามิเตอร์:
 - $\delta = 0.01$ (ระดับนัยสำคัญ)
 - $\text{min_fold_len} = 15$ (ความยาวขั้นต่ำของแต่ละช่วง)

3.4.2 ขั้นตอนการตรวจจับ

- วิเคราะห์ข้อมูลตามลำดับเวลา
- เมื่อตรวจจับ drift:
 - ตรวจสอบว่าช่วงข้อมูลก่อน drift มีข้อมูลเพียงพอสำหรับ train/test
 - ถ้าพอ เก็บตำแหน่ง drift point
 - ถ้าไม่พอ ข้าม drift point นั้น
- ตรวจสอบช่วงสุดท้าย (หลัง drift point สุดท้ายถึงจุดสิ้นสุดข้อมูล)
- ถ้าข้อมูลไม่พอ ลบ drift point สุดท้ายออก
 - ผลลัพธ์: รายการตำแหน่ง (index) ที่พบ concept drift

3.5 แบบจำลองที่ใช้ในการทำนาย

3.5.1 โครงสร้างแบบจำลอง ใช้ 3 ประเภท

- Recurrent Neural Network (RNN)
- LSTM (Long Short-Term Memory)
- GRU (Gated Recurrent Unit)

โครงสร้างเครือข่าย

Input Layer → RNN Layer 1 (32 units) → Dropout (0.2)

→ RNN Layer 2 (16 units) → Dropout (0.2)

→ RNN Layer 3 (8 units) → Dropout (0.2)

→ Dense Layer (16 units, ReLU)

→ Output Layer (1 unit)

3.5.2 การเตรียมข้อมูลสำหรับ RNN

ในขั้นตอนการเตรียมข้อมูลสำหรับการสร้างแบบจำลอง ได้มีการจัดรูปแบบข้อมูลให้อยู่ในลักษณะของ ลำดับเวลา (sequence data) โดยใช้ข้อมูลย้อนหลังจำนวน 15 วันเป็นตัวแปรนำเข้า เพื่อใช้ในการพยากรณ์ ค่าของวันถัดไป ตัวอย่างเช่น ใช้ข้อมูลในช่วงวันที่ 1–15 เพื่อทำนายค่าของวันที่ 16 และใช้ข้อมูลในช่วงวันที่ 2–16 เพื่อทำนายค่าของวันที่ 17 โดยเลื่อนช่วงข้อมูลไปตามลำดับเวลา

นอกจากนี้ เพื่อให้ข้อมูลอยู่ในช่วงที่เหมาะสมและช่วยเพิ่มประสิทธิภาพในการเรียนรู้ของแบบจำลอง ได้ทำการปรับสเกลข้อมูลด้วยวิธี StandardScaler ซึ่งช่วยทำให้ข้อมูลมีค่าเฉลี่ยเป็นศูนย์และมีส่วนเบี่ยงเบน มาตรฐานเท่ากับหนึ่ง

3.5.3 พารามิเตอร์การฝึก

- sequence_length = 15
- units = 32 (layer แรก)
- dropout_rate = 0.2
- learning_rate = 0.001
- epochs = 50
- batch_size = 32
- ใช้ Early Stopping (patience = 20)

3.5.4 Linear Regression

ในการสร้างแบบจำลองพยากรณ์ ได้เลือกใช้วิธี Linear Regression จากไลบรารี scikit-learn ซึ่งเป็น แบบจำลองเชิงเส้นพื้นฐานสำหรับการวิเคราะห์ความสัมพันธ์ระหว่างตัวแปร โดยก่อนนำข้อมูลเข้าสู่ แบบจำลอง ได้มีการปรับสเกลข้อมูลด้วยวิธี StandardScaler เพื่อให้ตัวแปรอยู่ในช่วงเดียวกันและช่วยลด ผลกระทบจากความแตกต่างของสเกลข้อมูล

ทั้งนี้ กระบวนการดังกล่าวไม่จำเป็นต้องจัดข้อมูลให้อยู่ในรูปแบบลำดับเวลา (sequence data) โดย ใช้ข้อมูลในแต่ละช่วงเวลานำเข้าสู่แบบจำลองโดยตรงเพื่อทำการพยากรณ์

3.6 กลยุทธ์การตรวจสอบความถูกต้อง (Cross-Validation Strategies)

3.6.1 Adaptive Cross-Validation

- แบ่งข้อมูลตาม drift points ที่ตรวจจับได้
- แต่ละช่วงแบ่งเป็น train (80%) และ test (20%)
- ตรวจสอบว่ามีข้อมูลเพียงพอสำหรับ train/test ก่อนประเมิน

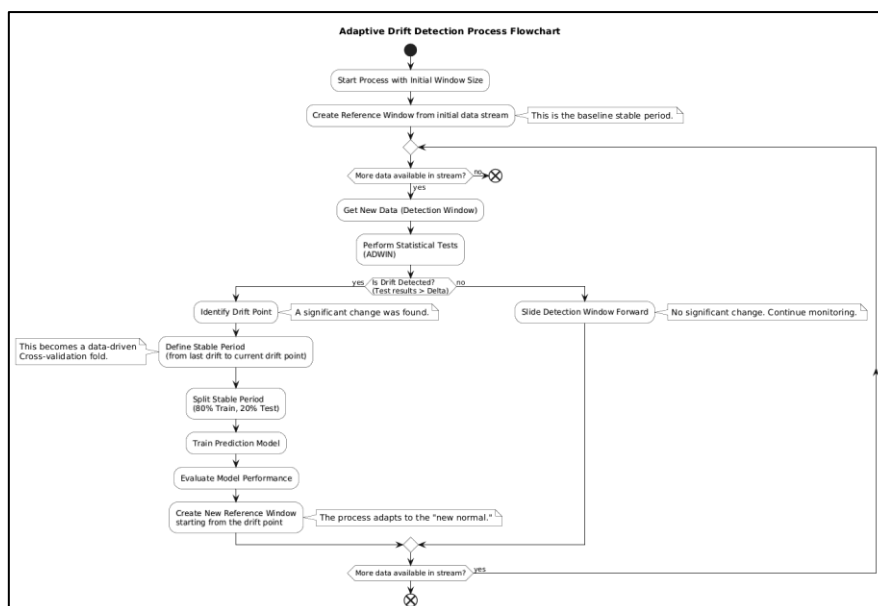
3.6.2 Baseline Cross-Validation

- Baseline 5-Fold
 - แบ่งข้อมูลเป็น 5 ส่วนเท่าๆ กัน
 - แต่ละส่วนแบ่งเป็น train (80%) และ test (20%)
- Baseline 10-Fold
 - แบ่งข้อมูลเป็น 10 ส่วนเท่าๆ กัน
 - แต่ละส่วนแบ่งเป็น train (80%) และ test (20%)
- Adaptive Baseline
 - ใช้จำนวน folds เท่ากับ Adaptive CV
 - แบ่งข้อมูลแบบเท่าๆ กัน (ไม่ใช่ drift points)

3.7 การวัดประสิทธิภาพของโมเดล

การประเมินผลใช้ตัวชี้วัดหลักจำนวน 2 ตัว คือ Root Mean Squared Error (RMSE) และ Mean Absolute Error (MAE) โดยในขั้นตอนการประเมิน โมเดลจะถูกทดสอบผ่านหลายชุดข้อมูล (Fold) ตามวิธี Cross-Validation ที่กำหนดไว้ โดยผลลัพธ์ของ RMSE และ MAE ที่ได้จากแต่ละ Fold จะถูกบันทึกและจัดเก็บอย่างเป็นระบบ เพื่อนำมาคำนวณค่าเฉลี่ยของแต่ละตัวชี้วัดในภาพรวม ซึ่งช่วยให้สามารถเปรียบเทียบประสิทธิภาพของโมเดลได้อย่างแม่นยำและมีความน่าเชื่อถือมากขึ้น ทั้งยังลดความลำเอียงที่อาจเกิดจากการแบ่งข้อมูลแบบสุ่มชุดเดียวเท่านั้น ทำให้ผลลัพธ์ที่ได้สะท้อนความสามารถของโมเดลในการทำนายข้อมูลจริงได้อย่างแท้จริง

3.8 ผังงานกระบวนการตรวจจับการเปลี่ยนแปลงแบบปรับตัว



ภาพที่ 9 ผังงานแสดงกระบวนการ Adaptive Drift Detection

จากภาพที่ 9 อธิบายถึงกระบวนการที่ทรงพลังและเชิงรุกในการจัดการ Model Machine Learning ในสภาพแวดล้อมจริงที่มีการเปลี่ยนแปลงอยู่เสมอ กระบวนการนี้ถูกเรียกว่า แบบปรับตัว (Adaptive) เนื่องจากมันไม่เพียงแค่ตรวจจับปัญหาเท่านั้น แต่ยังสามารถปรับเปลี่ยนและฝึกฝนตัวเองใหม่โดยอัตโนมัติเพื่อรับมือกับรูปแบบข้อมูลใหม่ที่เกิดขึ้น มีการทำงานแบบทีละขั้นตอน (Step-by-Step Breakdown) ดังนี้

ขั้นตอนที่ 1: การเตรียมความพร้อม (Initialization)

- กำหนดขนาดหน้าต่างเริ่มต้น: เริ่มต้นกระบวนการด้วยการกำหนดจำนวนข้อมูลที่จะใช้เป็นหน้าต่างอ้างอิง (Reference Window)
- สร้างหน้าต่างอ้างอิง: ใช้ข้อมูลชุดแรกเพื่อสร้าง หน้าต่างอ้างอิง (Reference Window) ซึ่งเป็นตัวแทนของข้อมูลที่เสถียร หรือ ปกติ

ขั้นตอนที่ 2: การเฝ้าระวัง (Monitoring)

- รับข้อมูลใหม่: ระบบจะรับข้อมูลเข้ามาอย่างต่อเนื่อง และนำไปจัดเก็บใน หน้าต่างตรวจจับ (Detection Window) ซึ่งจะอยู่ถัดจากหน้าต่างอ้างอิง
- เปรียบเทียบ: ใช้การทดสอบทางสถิติ (เช่น ADWIN) เพื่อเปรียบเทียบข้อมูลใน หน้าต่างตรวจจับ กับ หน้าต่างอ้างอิง เพื่อหาความแตกต่าง
- ประเมินผล
 - ถ้าไม่พบความแตกต่าง: ระบบจะสรุปว่าข้อมูลยังคงปกติ และจะ เลื่อนหน้าต่างตรวจจับ ไปข้างหน้าเพื่อเฝ้าระวังข้อมูลใหม่ในรอบถัดไป
 - ถ้าพบความแตกต่าง: ระบบจะระบุว่าเกิด "Data Drift" ขึ้น และจะเข้าสู่ขั้นตอนการปรับตัว

ขั้นตอนที่ 3: การปรับตัว (Adaptation)

- ระบุจุดเปลี่ยน: ระบบจะหา "จุดที่ข้อมูลเริ่มเปลี่ยน" (Drift Point) ที่แน่นอน ซึ่งบ่งชี้ถึงจุดที่ข้อมูลเริ่มไม่สอดคล้องกับหน้าต่างอ้างอิงเดิม
- กำหนดช่วงข้อมูลใหม่: กำหนดช่วงข้อมูลที่เสถียรชุดใหม่ (Stable Period) โดยเริ่มจาก จุดที่ข้อมูลเริ่มเปลี่ยน ไปจนถึงข้อมูลล่าสุดที่ได้รับ
- ฝึกโมเดลใหม่: นำข้อมูลใน ช่วงข้อมูลที่เสถียรใหม่ มาใช้ในการฝึกอบรม โมเดลพยากรณ์ใหม่ โดยปกติ จะมีการแบ่งข้อมูลเป็นส่วนใหญ่สำหรับฝึก (80%) และส่วนสำหรับทดสอบ (20%)

ขั้นตอนที่ 4: การอัปเดต (Update)

- ประเมินผลลัพธ์: ทดสอบประสิทธิภาพของโมเดลที่ฝึกใหม่ เพื่อให้แน่ใจว่าทำงานได้ดีบนข้อมูลที่เปลี่ยนไปแล้ว
- สร้างหน้าต่างอ้างอิงใหม่: เมื่อโมเดลใหม่พร้อมใช้งาน ระบบจะ สร้าง "หน้าต่างอ้างอิง" ใหม่ โดยเริ่มต้นจาก จุดที่ข้อมูลเริ่มเปลี่ยน
- กลับไปขั้นตอนที่ 2: ระบบจะกลับไปสู่ขั้นตอนการเฝ้าระวังอีกครั้ง แต่จะใช้หน้าต่างอ้างอิงที่อัปเดตแล้ว เพื่อให้การตรวจจับในอนาคตมีความแม่นยำและสอดคล้องกับสภาพข้อมูลปัจจุบัน

3.9 เครื่องมือและซอฟต์แวร์

การวิจัยครั้งนี้ดำเนินการโดยใช้เครื่องมือและซอฟต์แวร์ที่ทันสมัย เพื่อสนับสนุนการประมวลผลข้อมูล การสร้างและประเมินโมเดล ดังนี้

- ภาษาโปรแกรม Python 3
- สภาพแวดล้อมการพัฒนา (Development Environment)
 - ใช้ Python 3.13.5 ผ่าน Command Line Interface (CLI) เพื่อรันโปรแกรมแบบ backend โดยไม่ต้องใช้ web framework
- ไลบรารีและแพ็คเกจสำคัญ
 - Pandas, NumPy: สำหรับจัดการข้อมูลและคำนวณเชิงตัวเลข
 - Scikit-learn: สำหรับแบ่งชุดข้อมูลแบบ Time Series Split และวัดประสิทธิภาพของโมเดล
 - TensorFlow และ Keras: สำหรับสร้างและฝึกโครงข่ายประสาทเทียม (RNN, LSTM, GRU)
 - SciPy: สำหรับการทดสอบทางสถิติที่ใช้ในการตรวจจับ Concept Drift
- ระบบปฏิบัติการและฮาร์ดแวร์ งานวิจัยนี้ดำเนินการบนระบบปฏิบัติการ Windows 64 บิต ที่รองรับสถาปัตยกรรม x64 โดยใช้คอมพิวเตอร์ส่วนบุคคลที่มีสเปค ดังนี้
 - ตัวประมวลผล (CPU): Intel(R) Core(TM) i5-10300H ความเร็ว 2.50 GHz
 - หน่วยความจำ (RAM): ขนาด 16.00 GB (ใช้งานได้ประมาณ 15.84 GB)

บทที่ 4

ผลการศึกษา

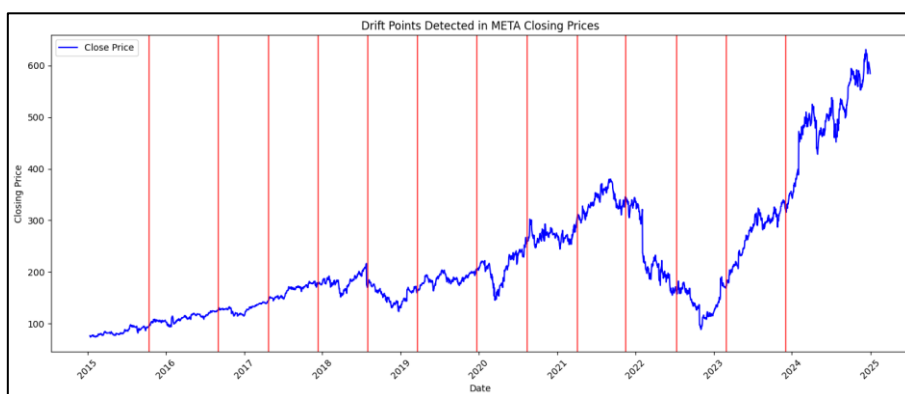
การทดลองในงานวิจัยนี้มีวัตถุประสงค์เพื่อนำเสนอประสิทธิภาพของโมเดลโครงข่ายประสาทเทียม RNN, LSTM, GRU และโมเดลเชิงเส้น (Linear Regression) ในการพยากรณ์ราคาหุ้น META NVIDIA และ TSLA ภายใต้เงื่อนไขที่มีการเปลี่ยนแปลงของข้อมูล โดยใช้แนวทาง Adaptive Cross-Validation ที่อิงจากตำแหน่งของ Drift Points และเปรียบเทียบกับวิธีการประเมินแบบ Baseline TimeSeriesSplit ซึ่งไม่พิจารณา Drift เพื่อให้บรรลุสำเร็จจุดประสงค์ของการวิจัยที่กำหนดไว้ ดังนี้

1. การตรวจจับ Concept Drift
2. ผลการทดลองด้วย Adaptive และ Baseline Cross-Validation
3. การเปรียบเทียบประสิทธิภาพของโมเดล
4. สรุปผลการวิเคราะห์

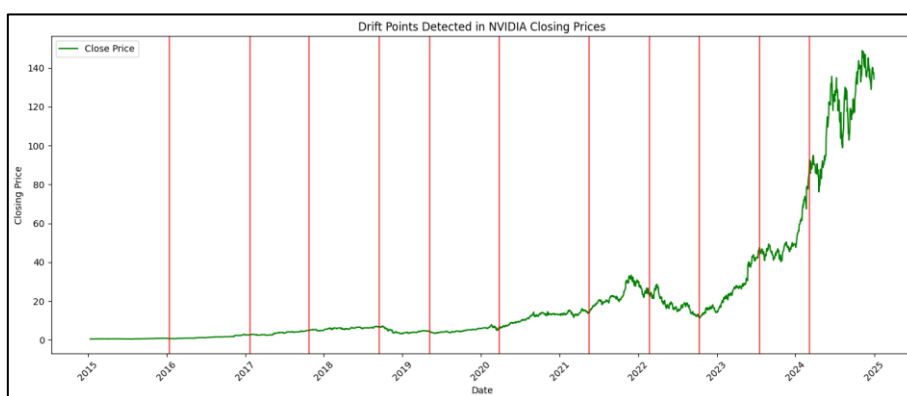
4.1 การตรวจจับ Concept Drift

- จุดเปลี่ยนแปลงสำคัญในข้อมูลหุ้นของบริษัท META ทั้งสิ้น 13 จุด ดังภาพที่ 10 ได้แก่
 - 14 ตุลาคม 2015 , 2 กันยายน 2016
 - 25 เมษายน 2017 , 11 ธันวาคม 2017
 - 1 สิงหาคม 2018 , 22 มีนาคม 2019
 - 24 ธันวาคม 2019 , 13 สิงหาคม 2020
 - 5 เมษายน 2021 , 18 พฤศจิกายน 2021
 - 12 กรกฎาคม 2022 , 1 มีนาคม 2023 และ 4 ธันวาคม 2023
- จุดเปลี่ยนแปลงสำคัญในข้อมูลหุ้นของบริษัท NVIDIA ทั้งสิ้น 11 จุด ดังภาพที่ 11 ได้แก่
 - 15 มกราคม 2016 , 23 มกราคม 2017
 - 25 ตุลาคม 2017 , 17 กันยายน 2018
 - 8 พฤษภาคม 2019 , 27 มีนาคม 2020
 - 19 พฤษภาคม 2021 , 22 กุมภาพันธ์ 2022
 - 11 ตุลาคม 2022 , 19 กรกฎาคม 2023 และ 7 มีนาคม 2024
- จุดเปลี่ยนแปลงสำคัญในข้อมูลหุ้นของบริษัท TSLA ทั้งสิ้น 12 จุด ดังภาพที่ 12 ได้แก่
 - 14 ตุลาคม 2015 , 20 กรกฎาคม 2016
 - 25 เมษายน 2017 , 11 ธันวาคม 2017
 - 1 สิงหาคม 2018 , 8 พฤษภาคม 2019
 - 11 กุมภาพันธ์ 2020 , 29 กันยายน 2020
 - 19 พฤษภาคม 2021 , 22 กุมภาพันธ์ 2022

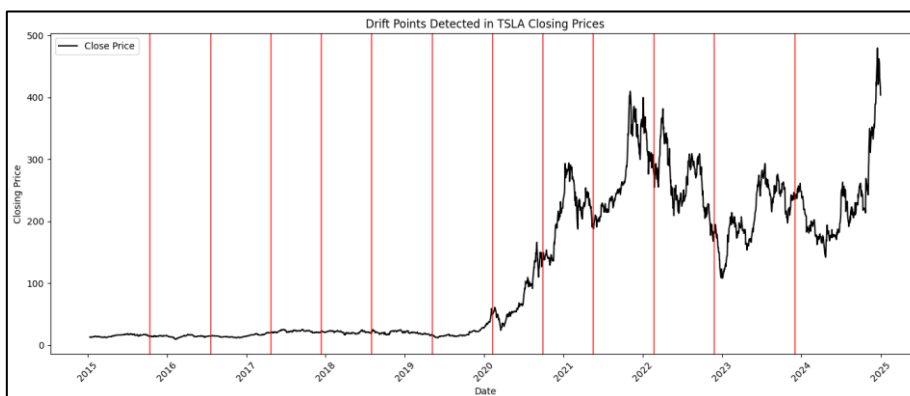
○ 25 พฤศจิกายน 2022 และ 4 ธันวาคม 2023



ภาพที่ 10 การตรวจจับ Concept Drift ของข้อมูลราคาหุ้น META



ภาพที่ 11 การตรวจจับ Concept Drift ของข้อมูลราคาหุ้น NVIDIA



ภาพที่ 12 การตรวจจับ Concept Drift ของข้อมูลราคาหุ้น TSLA

4.2 ผลการทดลองด้วย Adaptive และ Baseline Cross-Validation

4.2.1 ผลการทดลองด้วย Adaptive และ Baseline Cross-Validation ของหุ่น META

ตารางที่ 1 ผลคลาดเคลื่อนแต่ละโฟลด์ของ Adaptive CV (META)

Model								
Fold	RNN		LSTM		GRU		Linear	
	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE
Fold 1	4.793	3.376	7.079	5.376	7.594	5.493	8.753	7.604
Fold 2	8.894	8.541	12.114	11.564	9.120	8.967	12.118	11.396
Fold 3	10.645	10.520	11.905	10.368	12.429	11.224	12.123	11.932
Fold 4	8.933	8.377	8.663	7.905	8.070	7.289	16.977	16.361
Fold 5	19.590	18.074	20.983	19.424	19.009	17.277	21.464	19.464
Fold 6	15.305	13.342	5.585	4.087	13.655	12.481	10.702	9.873
Fold 7	15.287	14.146	20.483	19.940	13.836	11.746	12.035	10.936
Fold 8	45.074	35.898	27.716	21.794	30.165	22.707	50.188	46.443
Fold 9	13.417	11.706	13.768	11.473	11.906	9.650	12.052	10.152
Fold 10	16.982	14.789	24.486	21.447	26.136	23.301	10.410	8.949
Fold 11	40.193	39.887	38.999	38.545	33.022	32.541	74.466	69.724
Fold 12	39.689	30.543	52.329	45.120	42.352	37.841	31.582	24.484
Fold 13	39.208	36.258	30.629	26.302	32.619	28.446	49.068	47.188
Fold 14	87.125	85.534	87.744	85.442	87.932	86.001	99.806	95.110
Average	26.081	23.642	25.892	23.485	24.846	22.497	30.125	27.830

ตารางที่ 2 ผลคลาดเคลื่อนแต่ละโฟลด์ของ Baseline-5 CV (META)

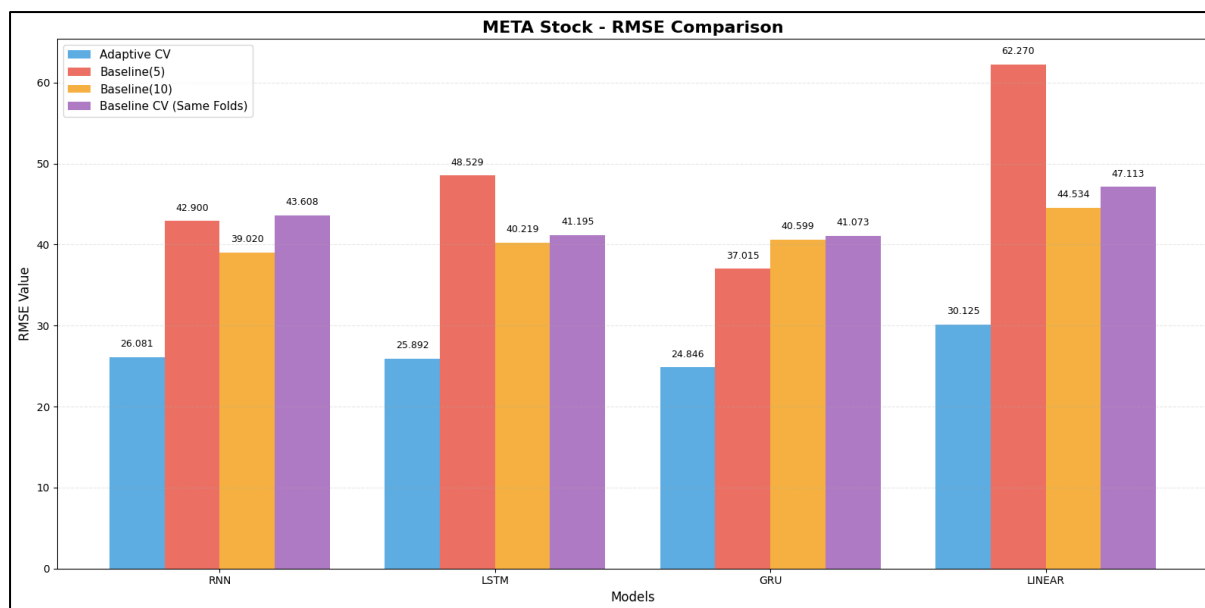
Model								
Fold	RNN		LSTM		GRU		Linear	
	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE
Fold 1	29.155	25.930	25.651	23.034	28.444	26.248	27.630	27.162
Fold 2	21.822	17.266	19.575	13.874	12.552	9.814	25.890	22.119
Fold 3	10.858	9.246	15.587	12.154	11.281	9.393	16.034	13.440
Fold 4	78.195	76.726	87.488	78.317	70.127	65.152	88.259	86.719
Fold 5	74.468	63.363	94.345	82.832	62.671	55.172	153.538	140.327
Average	42.900	38.506	48.529	42.042	37.015	33.156	62.270	57.953

ตารางที่ 3 ผลคลาดเคลื่อนแต่ละโพล์ของ Baseline-10 CV (META)

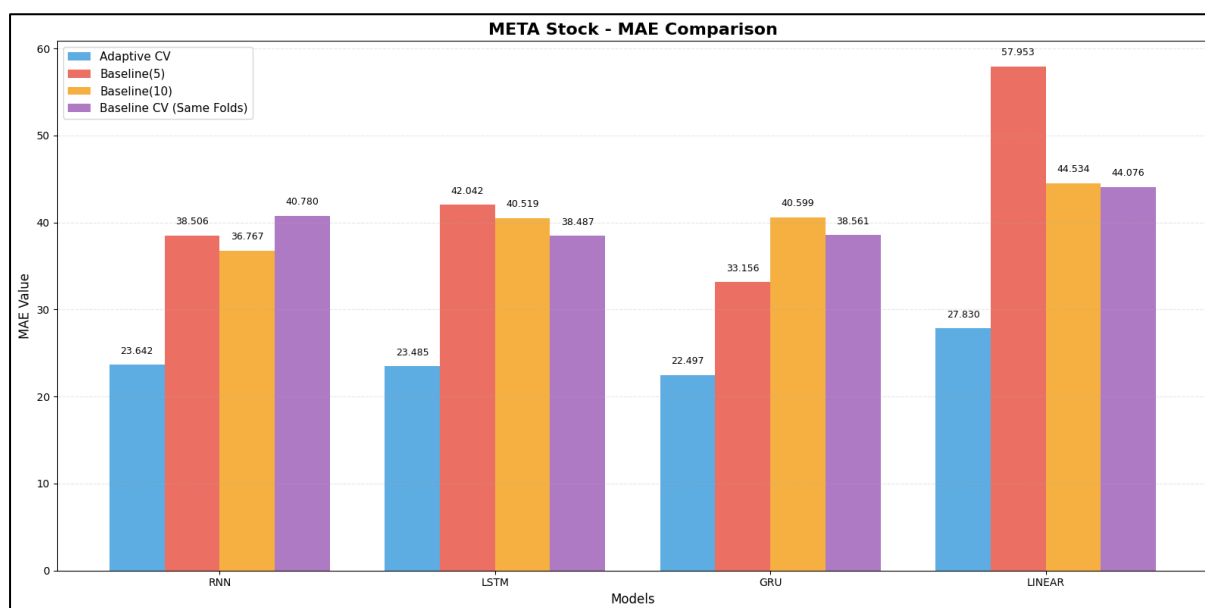
Model								
Fold	RNN		LSTM		GRU		Linear	
	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE
Fold 1	14.053	13.566	13.511	13.283	15.753	14.957	17.866	16.885
Fold 2	9.919	9.806	8.678	8.366	12.563	11.697	13.365	13.092
Fold 3	19.061	18.734	18.804	18.462	17.486	17.198	30.336	29.975
Fold 4	14.386	11.291	16.406	12.602	14.979	12.160	18.908	15.177
Fold 5	26.922	24.430	27.370	25.038	20.060	18.650	25.010	22.529
Fold 6	32.719	29.586	32.249	25.774	30.432	24.682	34.833	30.221
Fold 7	36.322	35.505	34.844	33.943	35.191	34.430	37.198	34.315
Fold 8	116.688	116.166	122.359	121.938	120.856	120.560	108.355	106.207
Fold 9	28.785	20.190	38.310	29.681	41.571	34.920	22.769	19.726
Fold 10	91.340	88.395	92.663	89.295	97.096	93.837	136.703	126.898
Average	39.020	36.767	40.519	37.838	40.599	38.309	44.534	41.503

ตารางที่ 4 ผลคลาดเคลื่อนแต่ละโพล์ของ Baseline CV (เท่ากับจำนวนโพล์ Adaptive) ของ META

Model								
Fold	RNN		LSTM		GRU		Linear	
	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE
Fold 1	4.092	2.864	4.949	4.183	6.181	5.541	11.233	10.247
Fold 2	11.831	11.797	12.132	12.018	9.918	9.760	12.498	12.235
Fold 3	9.649	9.358	11.126	10.917	10.011	9.833	9.099	8.616
Fold 4	10.338	9.371	8.598	7.655	9.032	7.287	16.127	14.146
Fold 5	20.728	19.032	20.538	19.194	20.343	19.155	21.390	19.316
Fold 6	13.814	11.532	8.755	7.554	15.515	14.455	13.039	11.740
Fold 7	14.396	13.365	19.683	18.545	18.198	17.180	12.577	11.387
Fold 8	89.389	87.677	82.467	81.801	75.111	74.493	73.934	69.606
Fold 9	38.612	38.117	40.435	35.364	35.948	35.422	42.019	41.087
Fold 10	80.488	65.663	80.804	66.517	78.960	64.407	57.130	42.718
Fold 11	62.515	54.925	82.905	80.730	82.530	79.667	57.176	53.840
Fold 12	78.849	76.356	65.757	64.403	56.604	54.608	103.654	97.314
Fold 13	77.075	73.912	44.120	42.733	74.440	73.091	147.512	147.145
Fold 14	98.738	96.950	94.459	87.205	82.237	74.960	82.201	77.662
Average	43.608	40.780	41.195	38.487	41.073	38.561	47.113	44.076



ภาพที่ 13 การเปรียบเทียบค่าเฉลี่ยของ RMSE ในแต่ละโมเดลของหุ้น META



ภาพที่ 14 การเปรียบเทียบค่าเฉลี่ยของ MAE ในแต่ละโมเดลของหุ้น META

4.2.2 ผลการทดลองด้วย Adaptive และ Baseline Cross-Validation ของหุ้น NVIDIA

ตารางที่ 5 ผลคลาดเคลื่อนแต่ละโฟลด์ของ Adaptive CV (NVIDIA)

Model								
Fold	RNN		LSTM		GRU		Linear	
	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE
Fold 1	0.252	0.249	0.256	0.255	0.256	0.253	0.233	0.227
Fold 2	1.107	1.066	0.873	0.836	0.786	0.770	0.942	0.882
Fold 3	1.230	1.094	0.697	0.640	0.981	0.927	1.312	1.276
Fold 4	0.590	0.457	0.722	0.555	0.804	0.644	0.761	0.709
Fold 5	0.738	0.669	0.467	0.442	0.356	0.311	0.418	0.320
Fold 6	1.901	1.731	1.969	1.866	1.738	1.643	2.406	2.258
Fold 7	1.163	0.987	1.094	0.884	0.984	0.836	1.498	1.216
Fold 8	6.647	6.485	4.292	3.906	4.908	4.504	4.042	3.375
Fold 9	4.339	4.311	4.446	4.420	3.357	3.328	4.175	3.805
Fold 10	17.778	17.632	17.039	16.831	25.542	25.522	17.762	16.997
Fold 11	28.946	28.333	26.313	25.625	25.47	24.563	23.869	22.791
Fold 12	11.698	10.595	16.658	13.477	10.590	8.281	15.283	12.427
Average	6.366	6.134	6.235	5.811	6.314	5.965	6.058	5.524

ตารางที่ 6 ผลคลาดเคลื่อนแต่ละโฟลด์ของ Baseline-5 CV (NVIDIA)

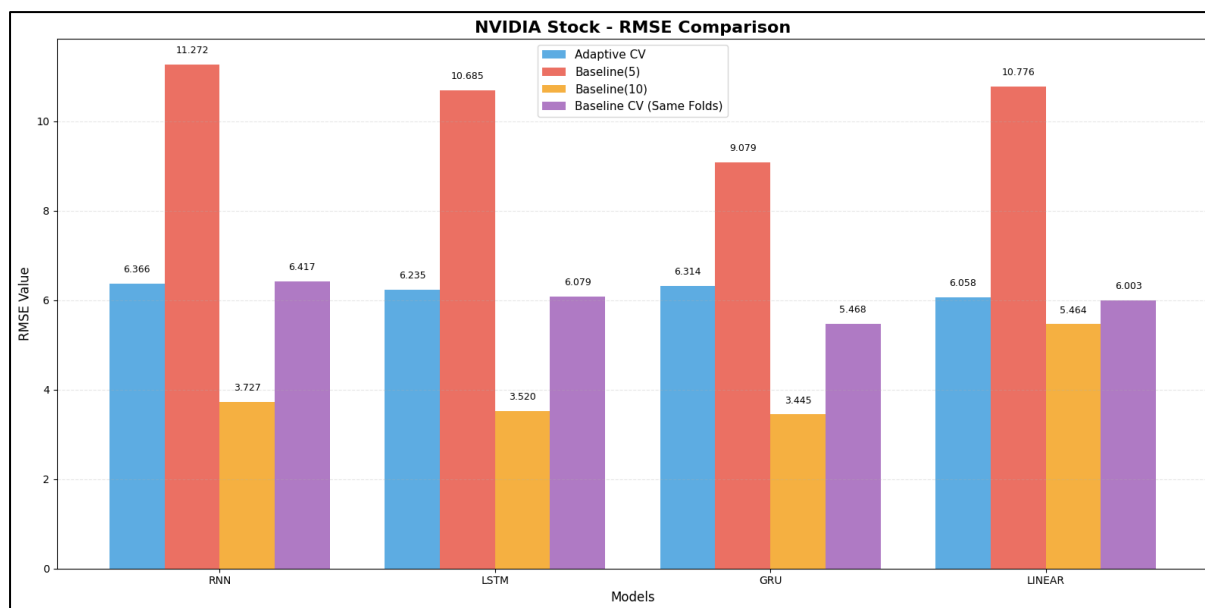
Model								
Fold	RNN		LSTM		GRU		Linear	
	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE
Fold 1	0.568	0.547	0.548	0.521	0.445	0.418	0.631	0.599
Fold 2	1.263	1.188	0.996	0.917	0.936	0.871	2.074	1.991
Fold 3	1.193	1.063	1.146	1.022	0.893	0.773	0.648	0.552
Fold 4	5.95	5.719	4.968	4.66	5.408	5.123	5.989	5.348
Fold 5	47.386	45.25	45.769	43.915	37.714	35.123	44.536	38.852
Average	11.272	10.753	10.685	10.207	9.079	8.462	10.776	9.469

ตารางที่ 7 ผลคลาดเคลื่อนแต่ละโฟลด์ของ Baseline-10 CV (NVIDIA)

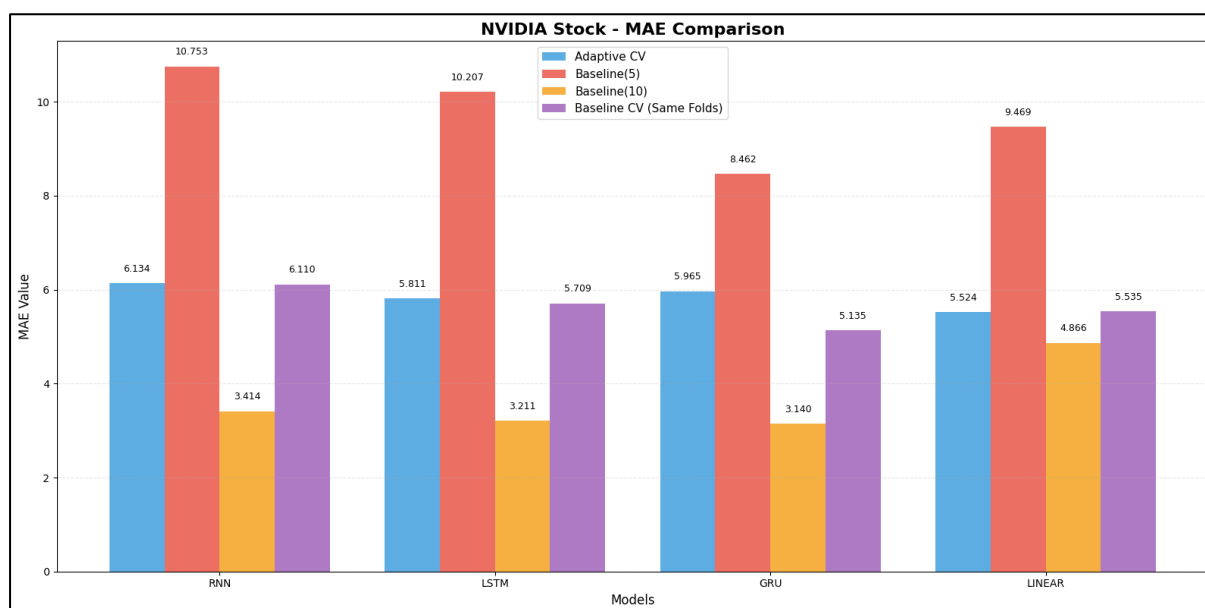
Model								
Fold	RNN		LSTM		GRU		Linear	
	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE
Fold 1	0.189	0.185	0.202	0.198	0.193	0.188	0.188	0.179
Fold 2	0.487	0.473	0.379	0.347	0.409	0.375	0.611	0.606
Fold 3	1.652	1.645	1.547	1.533	1.616	1.611	1.388	1.374
Fold 4	0.980	0.829	0.830	0.700	0.411	0.335	0.663	0.616
Fold 5	1.457	1.318	1.695	1.635	1.938	1.901	1.298	1.189
Fold 6	3.326	3.189	3.425	3.31	3.176	3.014	2.682	2.552
Fold 7	1.293	1.125	0.877	0.769	0.881	0.759	1.367	1.140
Fold 8	4.783	4.532	5.080	4.637	5.039	4.720	3.047	2.593
Fold 9	4.524	4.268	4.829	4.365	3.806	3.38	4.229	3.516
Fold 10	18.579	16.573	16.331	14.618	16.987	15.121	39.168	34.899
Average	3.727	3.414	3.520	3.211	3.445	3.140	5.464	4.866

ตารางที่ 8 ผลคลาดเคลื่อนแต่ละโฟลด์ของ Baseline CV (เท่ากับจำนวนโฟลด์ Adaptive) ของ NVIDIA

Model								
Fold	RNN		LSTM		GRU		Linear	
	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE
Fold 1	0.127	0.122	0.133	0.128	0.132	0.124	0.120	0.108
Fold 2	0.468	0.461	0.514	0.485	0.497	0.481	0.57	0.561
Fold 3	1.140	1.132	1.151	1.143	1.103	1.098	1.059	0.995
Fold 4	0.556	0.430	0.540	0.458	0.511	0.416	0.807	0.705
Fold 5	1.697	1.332	2.111	1.744	1.487	1.136	1.665	1.599
Fold 6	1.246	1.218	1.047	1.023	0.896	0.886	1.169	1.130
Fold 7	4.988	4.31	3.037	2.650	2.467	2.143	4.641	4.575
Fold 8	6.146	6.121	6.091	6.052	6.564	6.496	5.714	5.583
Fold 9	8.161	7.996	6.774	6.707	6.387	6.281	8.469	8.339
Fold 10	11.124	10.847	9.529	9.349	10.247	10.034	10.161	9.999
Fold 11	27.567	26.46	26.792	25.879	25.952	24.691	22.147	20.114
Fold 12	13.786	12.885	15.224	12.892	9.377	7.840	15.516	12.714
Average	6.417	6.110	6.079	5.709	5.468	5.135	6.003	5.535



ภาพที่ 15 การเปรียบเทียบค่าเฉลี่ยของ RMSE ในแต่ละโมเดลของหุ้น NVIDIA



ภาพที่ 16 การเปรียบเทียบค่าเฉลี่ยของ MAE ในแต่ละโมเดลของหุ้น NVIDIA

4.2.3 ผลการทดลองด้วย Adaptive และ Baseline Cross-Validation ของหุ้น TSLA

ตารางที่ 9 ผลคลาดเคลื่อนแต่ละโฟลด์ของ Adaptive CV (TSLA)

Model								
Fold	RNN		LSTM		GRU		Linear	
	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE
Fold 1	1.591	1.432	1.284	1.178	1.561	1.283	1.396	1.227
Fold 2	1.236	1.041	1.519	1.279	1.698	1.462	0.505	0.372
Fold 3	4.048	3.830	3.096	2.788	2.490	2.350	3.652	3.328
Fold 4	2.315	2.239	2.226	1.815	1.655	1.368	2.115	2.045
Fold 5	1.643	1.511	1.991	1.834	1.880	1.714	1.837	1.391
Fold 6	1.882	1.704	1.534	1.389	1.277	1.123	3.068	2.949
Fold 7	23.526	22.618	26.853	25.752	21.657	20.265	15.529	14.787
Fold 8	33.371	31.542	36.075	34.392	30.272	28.361	47.570	42.618
Fold 9	32.994	28.553	45.715	44.714	12.237	10.422	31.960	26.555
Fold 10	37.453	28.017	40.365	38.128	26.228	21.893	41.859	33.368
Fold 11	54.169	51.528	63.119	61.896	52.572	50.581	61.589	58.097
Fold 12	31.431	26.352	30.635	26.881	30.100	25.160	39.728	33.877
Fold 13	144.729	135.963	149.629	138.474	137.643	129.309	135.649	115.595
Average	28.491	25.872	31.080	29.271	24.713	22.715	29.727	25.862

ตารางที่ 10 ผลคลาดเคลื่อนแต่ละโฟลด์ของ Baseline-5 CV (TSLA)

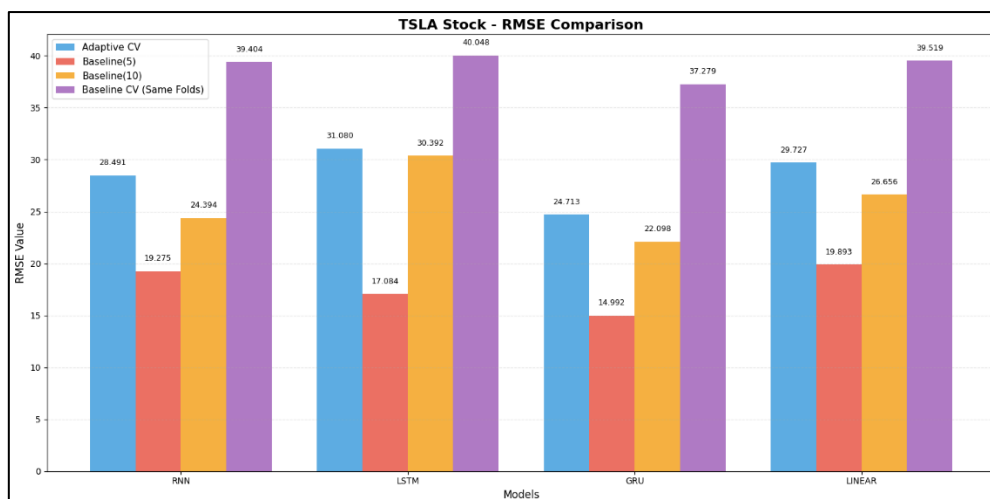
Model								
Fold	RNN		LSTM		GRU		Linear	
	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE
Fold 1	1.531	1.338	1.863	1.632	1.439	1.262	0.873	0.755
Fold 2	2.198	1.767	1.943	1.484	2.324	1.664	3.753	2.845
Fold 3	3.082	2.408	2.549	2.023	2.476	1.965	4.359	3.730
Fold 4	47.930	40.742	45.365	39.584	35.999	30.126	29.690	24.502
Fold 5	41.634	30.705	33.698	25.292	32.723	25.097	60.792	51.826
Average	19.275	15.392	17.084	14.003	14.992	12.023	19.893	16.731

ตารางที่ 11 ผลการทดสอบแต่ละโฟลด์ของ Baseline-10 CV (TSLA)

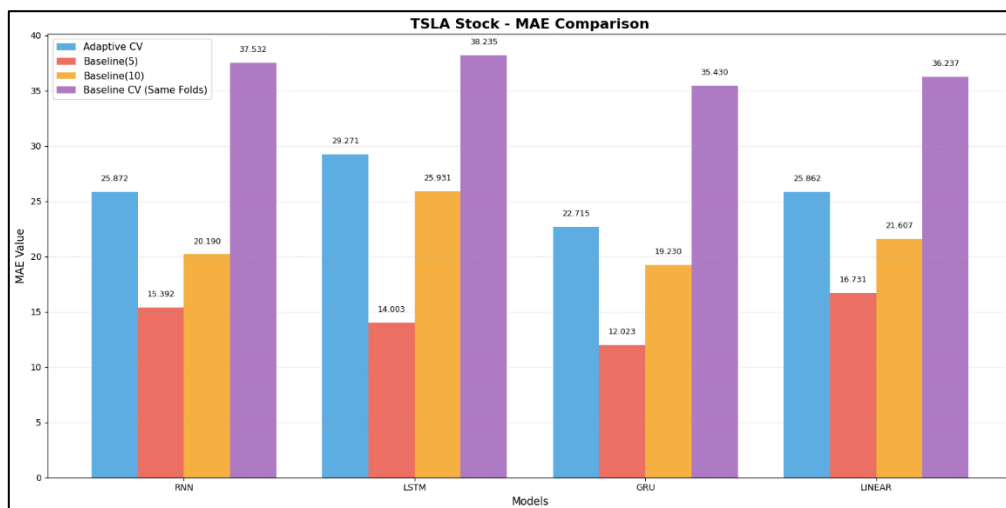
Model								
Fold	RNN		LSTM		GRU		Linear	
	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE
Fold 1	1.933	1.764	1.501	1.340	1.481	1.319	1.339	1.193
Fold 2	1.637	1.501	1.132	1.033	1.384	1.322	1.364	1.285
Fold 3	7.171	6.709	5.790	4.724	5.148	4.202	5.347	4.899
Fold 4	2.565	2.156	2.611	2.110	2.273	1.921	1.778	1.337
Fold 5	2.498	2.398	1.019	0.913	2.020	1.981	4.870	4.742
Fold 6	23.736	19.570	24.521	22.064	25.506	24.057	25.008	23.262
Fold 7	26.100	23.707	40.132	38.880	20.785	18.376	40.890	34.609
Fold 8	41.094	30.410	31.641	23.990	29.949	26.138	40.485	31.049
Fold 9	32.008	28.982	52.969	47.739	41.861	36.159	40.460	34.874
Fold 10	105.195	84.701	142.604	116.521	90.573	76.829	105.022	78.821
Average	24.394	20.190	30.392	25.931	22.098	19.230	26.656	21.607

ตารางที่ 12 ผลการทดสอบแต่ละโฟลด์ของ Baseline CV (เท่ากับจำนวนโฟลด์ Adaptive) ของ TSLA

Model								
Fold	RNN		LSTM		GRU		Linear	
	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE
Fold 1	1.361	1.177	1.510	1.274	1.119	0.846	1.409	1.230
Fold 2	1.352	1.100	1.389	1.223	1.732	1.395	0.503	0.375
Fold 3	3.797	3.726	2.515	2.396	2.969	2.859	4.081	3.773
Fold 4	1.196	0.923	0.961	0.841	0.917	0.833	0.964	0.774
Fold 5	2.562	2.191	2.953	2.417	1.972	1.761	2.311	1.871
Fold 6	4.127	3.430	3.533	3.210	3.679	3.579	4.114	3.786
Fold 7	8.166	6.812	10.624	9.348	11.289	9.349	17.656	14.451
Fold 8	54.628	51.024	57.989	53.841	58.657	54.965	91.086	84.513
Fold 9	137.582	136.000	151.913	150.797	135.804	134.173	126.041	116.780
Fold 10	50.037	45.658	44.294	39.180	32.592	26.192	24.761	20.131
Fold 11	33.878	27.163	31.517	25.730	26.996	21.674	39.343	32.409
Fold 12	49.089	48.623	49.608	48.842	47.370	46.714	60.285	59.384
Fold 13	164.471	160.084	161.820	157.952	159.525	156.249	141.194	131.602
Average	39.404	37.532	40.048	38.235	37.279	35.430	39.519	36.237



ภาพที่ 17 การเปรียบเทียบค่าเฉลี่ยของ RMSE ในแต่ละโมเดลของหุ้น TSLA



ภาพที่ 18 การเปรียบเทียบค่าเฉลี่ยของ MAE ในแต่ละโมเดลของหุ้น TSLA

4.3 สรุปผลการวิเคราะห์

การประเมินประสิทธิภาพของโมเดลสำหรับการพยากรณ์ข้อมูลอนุกรมเวลาที่มีความผันผวนและมีความเป็นไปได้อันจะเกิดการเปลี่ยนแปลงเชิงโครงสร้าง (Concept Drift) จำเป็นต้องใช้วิธีการตรวจสอบค่าความคลาดเคลื่อนที่เหมาะสม งานวิจัยนี้ได้ใช้สองวิธีในการตรวจสอบ ได้แก่ Adaptive Cross-Validation และ Baseline Cross-Validation เพื่อนำมาเปรียบเทียบผลลัพธ์ที่ได้จากโมเดลต่าง ๆ อันประกอบด้วย RNN, LSTM, GRU และ Linear บนข้อมูลราคาหุ้นของ NVIDIA, META และ TSLA

4.3.1 ผลลัพธ์ภายใต้ Adaptive Cross-Validation

Adaptive Cross-Validation เป็นวิธีการประเมินแบบจำลองที่ออกแบบมาเพื่อรองรับข้อมูลอนุกรมเวลาที่มีการเปลี่ยนแปลงของโครงสร้างข้อมูล (Concept Drift) โดยการแบ่งข้อมูลฝึกและทดสอบจะไม่อิงกับช่วงเวลาแบบตายตัว แต่จะอิงจากจุดที่ระบบตรวจพบว่าคุณสมบัติของข้อมูลเปลี่ยนไปอย่างมีนัยสำคัญผ่านอัลกอริทึม ADWIN

จากผลการทดลองพบว่า เมื่อใช้ Adaptive Cross-Validation ค่า MAE และ RMSE ของแบบจำลองในแต่ละ Fold มีความสม่ำเสมอมากขึ้น และมีแนวโน้มต่ำกว่าวิธี Baseline Cross-Validation โดยเฉพาะในช่วงเวลาที่ข้อมูลราคาหุ้นมีความผันผวนสูงหรือมีการเปลี่ยนแปลงแนวโน้มอย่างชัดเจน เช่น การเปลี่ยนจากตลาดขาขึ้นเป็นขาลง หรือช่วงที่ปริมาณการซื้อขายเพิ่มขึ้นผิดปกติ

สาเหตุสำคัญของผลลัพธ์ดังกล่าว คือการที่ Adaptive CV ทำให้ชุดข้อมูลฝึก (Training Set) และชุดข้อมูลทดสอบ (Test Set) อยู่ภายใต้บริบทเดียวกัน กล่าวคือ ข้อมูลทั้งสองชุดมีโครงสร้างและพฤติกรรมของตลาดที่ใกล้เคียงกันมากกว่าการแบ่งข้อมูลแบบคงที่ ส่งผลให้แบบจำลองไม่ต้องเรียนรู้จากข้อมูลในอดีตที่อาจไม่สอดคล้องกับสถานการณ์ปัจจุบัน

เมื่อพิจารณาในระดับของแบบจำลอง พบว่าแบบจำลองเชิงลำดับ ได้แก่ RNN, LSTM และ GRU มีประสิทธิภาพเหนือกว่า Linear Regression อย่างสม่ำเสมอ โดยเฉพาะ LSTM และ GRU ซึ่งให้ค่าความคลาดเคลื่อนต่ำที่สุดในหลายช่วงการทดลอง เนื่องจากแบบจำลองเหล่านี้มีกลไกหน่วยความจำ (Memory Mechanism) ที่สามารถเก็บและใช้ข้อมูลในอดีตระยะสั้นได้อย่างเหมาะสมกับแต่ละช่วง Drift โดยไม่พึ่งพาข้อมูลระยะยาวที่อาจล้าสมัย

กล่าวได้ว่า Adaptive Cross-Validation ช่วยจำกัดขอบเขตของอดีต ที่นำมาใช้ในการเรียนรู้ให้สอดคล้องกับสภาพข้อมูลจริงในแต่ละช่วงเวลา ทำให้การประเมินประสิทธิภาพของแบบจำลองมีความสมจริงมากยิ่งขึ้น

4.3.2 ผลลัพธ์ภายใต้ Baseline Cross-Validation

Baseline Cross-Validation เป็นวิธีการประเมินแบบจำลองที่ใช้กันอย่างแพร่หลาย โดยแบ่งข้อมูลออกเป็นหลาย Fold ตามลำดับเวลาแบบคงที่ เช่น 5-Fold หรือ 10-Fold โดยมีสมมติฐานสำคัญว่าการแจกแจงของข้อมูลมีความคงที่ตลอดเวลา

อย่างไรก็ตาม จากผลการทดลองพบว่า เมื่อใช้ Baseline Cross-Validation ค่า MAE และ RMSE ของแบบจำลองมีความผันผวนระหว่าง Fold ค่อนข้างสูง โดยเฉพาะในข้อมูลราคาหุ้นที่มี Concept Drift เกิดขึ้นบ่อยครั้ง ผลลัพธ์ในบาง Fold ให้ค่าความคลาดเคลื่อนต่ำมาก ขณะที่บาง Fold ให้ค่าความคลาดเคลื่อนสูงผิดปกติ

สาเหตุของปัญหานี้เกิดจากการที่ Baseline CV อาจนำข้อมูลจากหลายบริบทหรือหลายแนวคิด (Multiple Concepts) มาปะปนกันอยู่ในชุดข้อมูลฝึกเดียวกัน เช่น ใช้ข้อมูลจากช่วงตลาดค่อนข้างนิ่งมาฝึก แล้วนำไปทดสอบกับช่วงที่ตลาดผันผวนสูง ส่งผลให้แบบจำลองไม่สามารถเรียนรู้รูปแบบที่เหมาะสมกับข้อมูลทดสอบได้อย่างมีประสิทธิภาพ

แม้ว่าแบบจำลองเชิงลำดับอย่าง LSTM และ GRU จะยังคงให้ผลลัพธ์ที่ดีกว่า Linear Regression แต่ประสิทธิภาพโดยรวมลดลงเมื่อเทียบกับการประเมินแบบ Adaptive Cross-Validation แสดงให้เห็นว่าข้อจำกัดหลักไม่ได้อยู่ที่ตัวแบบจำลองเพียงอย่างเดียว แต่เกิดจากวิธีการแบ่งข้อมูลที่ไม่สอดคล้องกับธรรมชาติของข้อมูลอนุกรมเวลา ผลลัพธ์นี้ชี้ให้เห็นว่า Baseline Cross-Validation อาจให้ภาพลวงตาเกี่ยวกับประสิทธิภาพของแบบจำลอง ทั้งในแง่ที่ดีเกินจริง หรือดูแย่เกินจริง ขึ้นอยู่กับว่าแต่ละ Fold บังเอิญครอบคลุมช่วงข้อมูลลักษณะใด

4.3.3 การเปรียบเทียบเชิงปริมาณและเชิงเหตุผล

จากการเปรียบเทียบผลการประเมินแบบจำลองภายใต้ Adaptive Cross-Validation และ Baseline Cross-Validation ทั้งในเชิงปริมาณและเชิงเหตุผล พบว่าความแตกต่างของผลลัพธ์มีความชัดเจนและสอดคล้องกับลักษณะของข้อมูลอนุกรมเวลาที่มีการเปลี่ยนแปลงของโครงสร้างข้อมูลอย่างต่อเนื่อง

ในเชิงปริมาณ Adaptive Cross-Validation ให้ค่า Mean Absolute Error (MAE) และ Root Mean Square Error (RMSE) ต่ำกว่าในภาพรวมเกือบทุกแบบจำลองและทุกชุดข้อมูล ได้แก่ หุ้น META, NVIDIA และ TSLA โดยเฉพาะในข้อมูลที่มีความผันผวนสูง เช่น หุ้น TSLA ซึ่งแสดงให้เห็นถึงความไวของผลการพยากรณ์ต่อการเปลี่ยนแปลงของข้อมูลอย่างชัดเจน นอกจากนี้ ค่าความคลาดเคลื่อนภายใต้ Adaptive Cross-Validation มีความสม่ำเสมอระหว่างแต่ละ Fold มากกว่า เมื่อเปรียบเทียบกับ Baseline Cross-Validation ซึ่งมีความแปรปรวนของค่าความคลาดเคลื่อนระหว่าง Fold ค่อนข้างสูง

ในเชิงเหตุผล ความแตกต่างดังกล่าวสามารถอธิบายได้จากกลไกการแบ่งข้อมูลของทั้งสองวิธี Adaptive Cross-Validation ใช้จุดที่ตรวจพบ Concept Drift เป็นเกณฑ์ในการแบ่งข้อมูล ทำให้ชุดข้อมูลฝึกและชุดข้อมูลทดสอบภายในแต่ละ Fold มีลักษณะการกระจายและโครงสร้างของข้อมูลที่ใกล้เคียงกัน ส่งผลให้การประเมินประสิทธิภาพของแบบจำลองสะท้อนบริบทของข้อมูลในช่วงเวลานั้นได้อย่างเหมาะสม

ในทางตรงกันข้าม Baseline Cross-Validation อาศัยการแบ่งข้อมูลตามช่วงเวลาแบบคงที่ โดยไม่คำนึงถึงการเปลี่ยนแปลงของการกระจายข้อมูล ทำให้ข้อมูลจากหลายบริบทหรือหลายแนวคิดอาจถูกรวมอยู่ใน Fold เดียวกัน ซึ่งเพิ่มความยากในการเรียนรู้ของแบบจำลอง และส่งผลให้ค่าความคลาดเคลื่อนมีความผันผวนหรือสูงกว่าความเป็นจริง

ดังนั้น ผลการเปรียบเทียบในงานวิจัยนี้สะท้อนให้เห็นว่า Adaptive Cross-Validation ไม่เพียงให้ผลลัพธ์ที่ดีกว่าในเชิงตัวเลข แต่ยังให้ผลการประเมินที่มีความสอดคล้องกับธรรมชาติของข้อมูลอนุกรมเวลามากกว่า ซึ่งช่วยเพิ่มความน่าเชื่อถือของกระบวนการประเมินแบบจำลองในเชิงระเบียบวิธี

4.4 สรุปผลการวิเคราะห์

จากผลการวิเคราะห์ทั้งหมดในบทนี้ สามารถสรุปได้ว่า Adaptive Cross-Validation ที่อิงจากการตรวจจับ Concept Drift ด้วยอัลกอริทึม ADWIN เป็นแนวทางที่เหมาะสมสำหรับการประเมินประสิทธิภาพของแบบจำลองพยากรณ์ข้อมูลอนุกรมเวลาในบริบทที่ข้อมูลมีการเปลี่ยนแปลงของโครงสร้างอย่างต่อเนื่อง

ผลการศึกษาแสดงให้เห็นว่า การใช้วิธี Baseline Cross-Validation ซึ่งไม่คำนึงถึงการเกิด Concept Drift อาจนำไปสู่การประเมินประสิทธิภาพของแบบจำลองที่ไม่สะท้อนสภาพการใช้งานจริง เนื่องจากการแบ่งข้อมูลอาจทำให้ชุดข้อมูลฝึกและชุดข้อมูลทดสอบมีบริบทที่แตกต่างกันอย่างมีนัยสำคัญ ส่งผลให้ค่าความคลาดเคลื่อนที่ได้มีความแปรปรวนสูงและขาดความเสถียร

ในขณะที่ Adaptive Cross-Validation ช่วยลดผลกระทบจากข้อมูลในอดีตที่ไม่สอดคล้องกับสถานการณ์ปัจจุบัน และทำให้การประเมินแบบจำลองสะท้อนลักษณะการเปลี่ยนแปลงของข้อมูลได้อย่างเหมาะสมมากขึ้น โดยเฉพาะเมื่อใช้ร่วมกับแบบจำลองเชิงลำดับ เช่น LSTM และ GRU ซึ่งมีความสามารถในการเรียนรู้ความสัมพันธ์เชิงเวลาได้ดี

โดยสรุป งานวิจัยนี้ชี้ให้เห็นว่า การพัฒนาระบบพยากรณ์ข้อมูลอนุกรมเวลาไม่ควรให้ความสำคัญเฉพาะกับการเลือกแบบจำลองที่มีความซับซ้อนหรือให้ค่าความคลาดเคลื่อนต่ำเพียงอย่างเดียว แต่ควรให้ความสำคัญกับการออกแบบกระบวนการประเมินแบบจำลองที่สอดคล้องกับธรรมชาติของข้อมูล ซึ่ง Adaptive Cross-Validation เป็นแนวทางที่ช่วยเพิ่มทั้งความแม่นยำและความน่าเชื่อถือของการพยากรณ์ในเชิงวิชาการและการประยุกต์ใช้งานจริง

บทที่ 5

สรุป อภิปรายผล และข้อเสนอแนะ

ในการดำเนินงานวิจัยเรื่อง การตรวจสอบค่าความคลาดเคลื่อนในข้อมูลแบบปรับตัวได้สำหรับการพยากรณ์อนุกรมเวลา มีวัตถุประสงค์เพื่อศึกษารูปแบบความคลาดเคลื่อนของข้อมูลภายใต้เงื่อนไขที่สามารถปรับเปลี่ยนได้ตามบริบทของอนุกรมเวลา รวมทั้งเพื่อเปรียบเทียบประสิทธิภาพของวิธีการที่นำมาใช้วิเคราะห์ความคลาดเคลื่อนดังกล่าว โดยผลการศึกษาที่ได้ สามารถจัดแบ่งเป็นหัวข้อสำคัญในการสรุปผลการวิจัยอย่างเป็นระบบ ดังต่อไปนี้

1. สรุปผลการวิจัย
2. อภิปรายผลการวิจัย
3. ข้อเสนอแนะ

5.1 สรุปผลการวิจัย

งานวิจัยนี้มีวัตถุประสงค์เพื่อพัฒนาแนวทางการตรวจสอบความถูกต้องของแบบจำลองพยากรณ์ข้อมูลอนุกรมเวลาในสภาวะที่ข้อมูลมีการเปลี่ยนแปลงของโครงสร้าง โดยนำแนวคิด Adaptive Cross-Validation ที่อิงจากการตรวจจับ Concept Drift ด้วยอัลกอริทึม ADWIN มาประยุกต์ใช้ในกระบวนการประเมินแบบจำลอง และเปรียบเทียบผลลัพธ์กับวิธี Baseline Cross-Validation ซึ่งเป็นวิธีที่ใช้กันทั่วไป

การทดลองดำเนินการบนข้อมูลราคาหุ้นรายวันของบริษัท META, NVIDIA และ TSLA ซึ่งเป็นข้อมูลอนุกรมเวลาที่มีความผันผวนและมีแนวโน้มการเปลี่ยนแปลงของพฤติกรรมราคาในระยะยาว ผลการศึกษพบว่า Adaptive Cross-Validation ให้ผลการประเมินที่มีค่าความคลาดเคลื่อนต่ำกว่าและมีความสม่ำเสมอมากกว่าวิธี Baseline Cross-Validation อย่างชัดเจน ทั้งในแง่ของค่า MAE และ RMSE

เมื่อพิจารณาในระดับแบบจำลอง พบว่าแบบจำลองเชิงลำดับ ได้แก่ RNN, LSTM และ GRU แสดงให้เห็นถึงศักยภาพในการเรียนรู้ความสัมพันธ์เชิงเวลาได้ดีกว่า Linear Regression โดยเฉพาะในบริบทของข้อมูลที่ไม่คงที่ ผลลัพธ์ดังกล่าวสะท้อนให้เห็นว่า การเลือกแบบจำลองที่เหมาะสมควรพิจารณาควบคู่กับลักษณะของข้อมูลและวิธีการประเมินแบบจำลอง

โดยสรุป ผลการวิจัยสนับสนุนสมมติฐานที่ตั้งไว้ว่า การประเมินแบบจำลองด้วย Adaptive Cross-Validation ที่คำนึงถึง Concept Drift สามารถเพิ่มความน่าเชื่อถือของผลการประเมิน และช่วยลดความคลาดเคลื่อนที่เกิดจากการแบ่งข้อมูลแบบคงที่ตามช่วงเวลา

5.2 อภิปรายผลการวิจัย

ผลการวิจัยในครั้งนี้สะท้อนให้เห็นถึงประเด็นสำคัญในงานพยากรณ์ข้อมูลอนุกรมเวลา กล่าวคือ ประสิทธิภาพของแบบจำลองไม่ควรถูกพิจารณาแยกขาดจากกระบวนการประเมินแบบจำลองที่ใช้ การใช้ Baseline Cross-Validation ซึ่งตั้งอยู่บนสมมติฐานว่าการแจกแจงของข้อมูลมีความคงที่ตลอดช่วงเวลา อาจไม่เหมาะสมกับข้อมูลจริงที่มีการเปลี่ยนแปลงของโครงสร้างอย่างต่อเนื่อง

การที่ Adaptive Cross-Validation ให้ผลลัพธ์ที่ดีกว่า สามารถอธิบายได้จากการที่กระบวนการแบ่งข้อมูลอิงจากจุดที่ตรวจพบ Concept Drift ทำให้ชุดข้อมูลฝึกและชุดข้อมูลทดสอบมีลักษณะการกระจายและบริบทของข้อมูลที่ใกล้เคียงกันมากกว่า ส่งผลให้การประเมินประสิทธิภาพของแบบจำลองสะท้อนสภาพการใช้งานจริงได้อย่างเหมาะสม

นอกจากนี้ ผลการทดลองยังแสดงให้เห็นว่า แม้แบบจำลองเชิงลำดับอย่าง LSTM และ GRU จะมีความสามารถในการเรียนรู้ความสัมพันธ์เชิงเวลาได้ดี แต่หากใช้ร่วมกับวิธีการประเมินที่ไม่คำนึงถึงการเปลี่ยนแปลงของข้อมูล ประสิทธิภาพของแบบจำลองอาจถูกบิดเบือนไปจากความเป็นจริงได้ ผลลัพธ์นี้ตอกย้ำแนวคิดที่ว่า การจัดการ Concept Drift เป็นองค์ประกอบสำคัญของระบบพยากรณ์ที่มีความน่าเชื่อถือ

ในเชิงระเบียบวิธี งานวิจัยนี้มีส่วนช่วยเติมเต็มช่องว่างของงานวิจัยเดิมที่มักให้ความสำคัญกับการพัฒนาแบบจำลองหรืออัลกอริทึมตรวจจับ Drift แยกจากกัน โดยแสดงให้เห็นว่าการผสานการตรวจจับ Concept Drift เข้ากับขั้นตอนการ Cross-Validation สามารถยกระดับคุณภาพของการประเมินแบบจำลองได้อย่างเป็นระบบ

อย่างไรก็ตาม งานวิจัยนี้ไม่ได้มุ่งเน้นการนำเสนอค่าความแม่นยำเชิงจุดของการพยากรณ์ในแต่ละช่วงเวลา แต่ให้ความสำคัญกับการเปรียบเทียบกระบวนการประเมินแบบจำลองในภาพรวม ดังนั้น ผลลัพธ์ที่ได้ควรถูกตีความในบริบทของความน่าเชื่อถือของการประเมินแบบจำลอง มากกว่าการเปรียบเทียบเชิงตัวเลขเพียงอย่างเดียว

5.3 ข้อเสนอแนะ

งานวิจัยในอนาคตสามารถขยายการศึกษาไปยังข้อมูลอนุกรมเวลาประเภทอื่นที่มีการเปลี่ยนแปลงของโครงสร้างข้อมูล เพื่อประเมินความเหมาะสมของ Adaptive Cross-Validation ในบริบทที่หลากหลายมากขึ้น นอกจากนี้ ควรพิจารณาเปรียบเทียบกับอัลกอริทึมตรวจจับ Concept Drift ประเภทอื่น และเพิ่มการทดสอบทางสถิติเพื่อยืนยันความแตกต่างของผลการประเมินอย่างเป็นระบบ อันจะช่วยเสริมความแข็งแกร่งของข้อสรุปในเชิงวิชาการและการประยุกต์ใช้งานจริง

บรรณานุกรม

- Alexey Tsymbal. (2004). *The problem of concept drift: Definitions and related work*. Technical Report TCD-CS-2004-15, Department of Computer Science, Trinity College Dublin. เข้าถึงได้จาก <https://www.scss.tcd.ie/publications/tech-reports/reports.04/TCD-CS-2004-15.pdf>
- Bifet, A., & Gavald', R. (2007). Learning from Time-Changing Data with Adaptive Windowing. *Proceedings of the 2007 SIAM International Conference on Data Mining* (pp. 443-448). Society for Industrial and Applied Mathematics.
- Dries, A., & Rückert, U. (2009). Adaptive concept drift detection. *Statistical Analysis and Data Mining*, 2(5-6), 311–327.
- Elwell, R., & Polikar, R. (2011). Incremental learning of concept drift in nonstationary environments. *IEEE Transactions on Neural Networks*, 22(10), 1517–1531.
- Gareth James, Daniela Witten, Trevor Hastie, และ Robert Tibshirani. (2021). *An Introduction to Statistical Learning: with Applications in R (2nd ed.)*. Springer.
- Ian Goodfellow, Yoshua Bengio, และ Aaron Courville. (2016). *DEEP LEARNING*.
- Jan Zenisek, Florian Holzinger, และ Michael Affenzeller. (2019). Machine learning based concept drift detection for predictive maintenance. *Computers & Industrial Engineering*(137). เข้าถึงได้จาก <https://doi.org/10.1016/j.cie.2019.106031>
- Joao Gama, Indre Zliobaite, Albert Bifet, Mykola Pechenizkiy, และ Abdelhamid Bouchachia. (2013). A Survey on Concept Drift Adaptation. *ACM Computing Surveys (CSUR)*, 46(4)(44). doi:<https://doi.org/10.1145/2523813>
- Junyoung Chung, Caglar Gulcehre, KyungHyun Cho, และ Yoshua Bengio. (2014). *Empirical evaluation of gated recurrent neural networks on sequence modeling*. เข้าถึงได้จาก arXiv preprint: <https://arxiv.org/abs/1412.3555>
- Klaus Greff, Rupesh K Srivastava, Jan Koutník, Bas R Steunebrink, และ Jürgen Schmidhuber. (2017). LSTM: A Search Space Odyssey. *IEEE Transactions on Neural Networks and Learning Systems*.

- Krawczyk, B., Minku, L., Gama, J., Stefanowski, J., & Wozniak, M. (2017). Ensemble learning for data stream analysis: A survey. *Information Fusion*, 37, 132–156.
- Lifan Zhao, และ Yanyan Shen. (2025). Proactive Model Adaptation Against Concept Drift for Online Time Series Forecasting. *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '25)*, Toronto, Canada.
doi:<https://doi.org/10.1145/3690624.3709210>
- Lobo, J., Del Ser, J., & Herrera, F. (2021). Lunar: Cellular automata for drifting data streams. *Information Sciences*, 543, 467–487.
- Lu, J., Liu, A., Dong, F., Gu, F., Gama, J., & Zhang, G. (2019). Learning under concept drift: A review. *IEEE Transactions on Knowledge and Data Engineering*, 31(12), 2346–2363.
- Maryam H Bahar, และ Hadeel Noori Saad. (2024). Adaptive windowing (ADWIN3) to learning from time-changing data stream. ใน *Intelligent Systems Design and Applications (ISDA 2023)* (หน้า 150–163). Springer Nature Switzerland. เข้าถึงได้จาก
https://doi.org/10.1007/978-3-031-64850-2_14
- Mouxiang Chen, Lefei Shen, Han Fu, Zhuo Li, Jianling Sun, และ Chenghao Liu. (2024). Calibration of Time-Series Forecasting: Detecting and Adapting Context-Driven Distribution Shift. *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '24)*, Barcelona, Spain.
doi:<https://doi.org/10.1145/3637528.3671926>
- Peter Bruce, Andrew Bruce, และ Peter Gedeck. (2020). *Practical Statistics for Data Scientists (2nd ed.)*. United States of America: O'Reilly Media.
- Rafal Jozefowicz, Wojciech Zaremba, และ Ilya Sutskever. (2015). An empirical exploration of recurrent network architectures. *Proceedings of the 32nd International Conference on Machine Learning (ICML 2015)*. Lille, France: JMLR: Workshop and Conference Proceedings, Volume 37.
- Rob J Hyndman, และ George Athanasopoulos. (2021). *Forecasting: Principles and Practice*. OTexts.

- Roberto Souto, Maior Barros, Silas Garrido T, และ Carvalho Santos. (2018). A large-scale comparison of concept drift detectors. *Information Sciences*(451–452), 348–370. เข้าถึงได้จาก <https://doi.org/10.1016/j.ins.2018.04.014>
- Sepp Hochreiter, และ Jurgen Schmidhuber. (1997). Long short-term memory. *Neural Computation*.
- Sergio Ramírez-Gallego, Bartosz Krawczyk, Salvador García, Michal Wozniak, และ Francisco Herrera. (2017). A survey on data preprocessing for data stream mining: Current status and future directions. *Neurocomputing*(239), 39–57.
- Song, Y., Lu, J., Lu, H., & Zhang, G. (2021). Learning data streams with changing distributions and temporal dependency. *IEEE Transactions on Neural Networks and Learning Systems*.
- Wibul Yuk, และ Jaree Thongkham. (2018). Comparison of Time Series Techniques for Predicting Gold and Oil Prices. *RMUTI JOURNAL Science and Technology*.
- Yuan Zhong, Hongyu Yang, Yanci Zhang, Ping Li, และ Cheng Ren. (2021). Long short-term memory self-adapting online random forests for evolving data stream regression. *Neurocomputing*, 457, 265-276.
- Ziyi Liu, Rakshitha Godahewa, Kasun Bandara, & Christoph Bergmeir. (2023). *Handling Concept Drift in Global Time Series*. arXiv preprint, Melbourne, Australia. Retrieved from <https://arxiv.org/abs/2304.01512>

ภาคผนวก

1. การเตรียมข้อมูลและตรวจสอบข้อมูลพื้นฐาน

1.1 META Stock

```
1 meta
```

✓ 0.0s

	Date	Close	High	Low	Open	Volume
0	2015-01-02	78.021965	78.499349	77.276057	78.151261	18177500
1	2015-01-05	76.768845	78.817603	76.440643	77.554535	26452200
2	2015-01-06	75.734520	77.166658	74.948829	76.808629	27399300
3	2015-01-07	75.734520	76.937917	75.406319	76.341192	22045300
4	2015-01-08	77.753441	77.803171	75.664900	76.321295	23961000
...	...	...	...	...	...	...
2511	2024-12-24	606.742920	606.982512	598.286985	601.721226	4726100
2512	2024-12-26	602.350220	605.295344	597.947554	604.476695	6081400
2513	2024-12-27	598.816101	600.852699	588.822678	598.416739	8084200
2514	2024-12-30	590.260254	595.950821	584.609660	587.774390	7025900
2515	2024-12-31	584.539795	592.985737	582.882511	591.288603	6019500

2516 rows x 6 columns

ภาพที่ 19 การนำเข้าข้อมูลของ META Stock

```
1 meta.info()
```

✓ 0.0s

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2516 entries, 0 to 2515
Data columns (total 6 columns):
#   Column  Non-Null Count  Dtype
---  -
0   Date    2516 non-null   datetime64[ns]
1   Close   2516 non-null   float64
2   High    2516 non-null   float64
3   Low     2516 non-null   float64
4   Open    2516 non-null   float64
5   Volume  2516 non-null   int64
dtypes: datetime64[ns](1), float64(4), int64(1)
memory usage: 118.1 KB
```

ภาพที่ 20 รายละเอียดของชุดข้อมูลดั้งเดิมของ META Stock

```
1 print(meta.isnull().sum())
2 print("Duplicates:", meta.duplicated().sum())
```

✓ 0.0s

```
Date      0
Close     0
High      0
Low       0
Open      0
Volume    0
dtype: int64
Duplicates: 0
```

ภาพที่ 21 ตรวจสอบจำนวนค่าว่าง และจำนวนแถวข้อมูลที่ซ้ำกันของ META Stock

```
1 meta.describe()
```

✓ 0.0s

	Date	Close	High	Low	Open	Volume
count	2516	2516.000000	2516.000000	2516.000000	2516.000000	2.516000e+03
mean	2019-12-31 19:03:31.764706048	221.131964	223.844916	218.289474	221.034913	2.288157e+07
min	2015-01-02 00:00:00	73.645981	74.421715	71.607154	73.636027	4.726100e+06
25%	2017-07-02 06:00:00	135.658276	137.050625	133.495126	135.275356	1.440050e+07
50%	2020-01-01 00:00:00	182.091034	184.189529	180.176549	182.061187	1.921050e+07
75%	2022-06-30 06:00:00	279.339996	283.676249	274.780038	277.644326	2.639298e+07
max	2024-12-31 00:00:00	631.122559	636.828510	625.666100	629.945518	2.323166e+08
std	NaN	121.281334	122.771001	119.759084	121.318070	1.489255e+07

ภาพที่ 22 รายละเอียดค่าทางสถิติเบื้องต้นของ META Stock

```
1 meta_fea.info()
```

✓ 0.0s

```
<class 'pandas.core.frame.DataFrame'>
Index: 2510 entries, 6 to 2515
Data columns (total 4 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Return           2510 non-null   float64
1   Volatility        2510 non-null   float64
2   Volume_Log        2510 non-null   float64
3   Return_Volume     2510 non-null   float64
dtypes: float64(4)
memory usage: 98.0 KB
```

ภาพที่ 23 ชุดข้อมูลคงเหลือหลังจากเพิ่ม Feature Engineering ของ META Stock

```
1 meta_fea.describe()
```

✓ 0.0s

	Return	Volatility	Volume_Log	Return_Volume
count	2510.000000	2510.000000	2510.000000	2510.000000
mean	0.001089	4.640482	16.812645	0.021006
std	0.023630	4.344635	0.486052	0.472311
min	-0.263901	0.265485	15.368611	-5.233710
25%	-0.009110	1.778613	16.482393	-0.182375
50%	0.000999	3.462906	16.769647	0.019862
75%	0.012235	6.036191	17.088494	0.241979
max	0.232824	47.502297	19.263612	4.691648

ภาพที่ 24 รายละเอียดค่าทางสถิติเบื้องต้นหลังจากเพิ่ม Feature Engineering ของ META Stock

1.2 NVIDIA Stock

```
1 nvidia
```

✓ 0.0s

	Date	Close	High	Low	Open	Volume
0	2/1/2015	0.483066	0.486665	0.475386	0.483066	113680000
1	5/1/2015	0.474906	0.484505	0.472747	0.483066	197952000
2	6/1/2015	0.460508	0.476106	0.460028	0.475626	197764000
3	7/1/2015	0.459308	0.467947	0.457868	0.463868	321808000
4	8/1/2015	0.476586	0.479466	0.464348	0.464588	283780000
...	...	...	...	...	...	...
2511	24/12/2024	140.197372	141.877094	138.627619	139.977407	105157000
2512	26/12/2024	139.907410	140.827275	137.707768	139.677451	116205600
2513	27/12/2024	136.987885	138.997570	134.688268	138.527645	170582600
2514	30/12/2024	137.467804	140.247354	133.998363	134.808230	167734700
2515	31/12/2024	134.268326	138.047730	133.808409	138.007728	155659200

2516 rows x 6 columns

ภาพที่ 25 การนำเข้าข้อมูลของ NVIDIA Stock

```
1 nvidia.info()
2
```

✓ 0.0s

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2516 entries, 0 to 2515
Data columns (total 6 columns):
#   Column  Non-Null Count  Dtype
---  ---
0   Date    2516 non-null   datetime64[ns]
1   Close   2516 non-null   float64
2   High    2516 non-null   float64
3   Low     2516 non-null   float64
4   Open    2516 non-null   float64
5   Volume  2516 non-null   int64
dtypes: datetime64[ns](1), float64(4), int64(1)
memory usage: 118.1 KB
```

ภาพที่ 26 รายละเอียดของชุดข้อมูลดั้งเดิมของ NVIDIA Stock

```
2 print(nvidia.isnull().sum())
3 print("Duplicated : ",nvidia.duplicated().sum())
```

✓ 0.0s

```
Date      0
Close     0
High      0
Low       0
Open      0
Volume    0
dtype: int64
Duplicated : 0
```

ภาพที่ 27 ตรวจสอบจำนวนค่าว่าง และจำนวนแถวข้อมูลซ้ำกันของ NVIDIA Stock

```
1 nvidia.describe()
```

✓ 0.0s

	Date	Close	High	Low	Open	Volume
count	2516	2516.000000	2516.000000	2516.000000	2516.000000	2.516000e+03
mean	2019-12-31 19:03:31.764706048	20.786881	21.164913	20.375351	20.789328	4.676480e+08
min	2015-01-02 00:00:00	0.459308	0.467947	0.454509	0.463628	5.244800e+07
25%	2017-07-02 06:00:00	3.545036	3.594863	3.449813	3.515635	3.055120e+08
50%	2020-01-01 00:00:00	6.482601	6.539911	6.338270	6.440887	4.151695e+08
75%	2022-06-30 06:00:00	21.228698	21.733422	20.806877	21.166934	5.640210e+08
max	2024-12-31 00:00:00	148.845734	152.854800	146.226298	149.315621	3.692928e+09
std	NaN	32.314417	32.916986	31.675142	32.351868	2.536131e+08

ภาพที่ 28 รายละเอียดค่าทางสถิติเบื้องต้นของ NVIDIA Stock

```
1 NVDIA_fea.info()
```

✓ 0.0s

```
<class 'pandas.core.frame.DataFrame'>
Index: 2510 entries, 6 to 2515
Data columns (total 4 columns):
#   Column          Non-Null Count  Dtype  
---  -
0   Return           2510 non-null   float64
1   Volatility        2510 non-null   float64
2   Volume_Log        2510 non-null   float64
3   Return_Volume     2510 non-null   float64
dtypes: float64(4)
memory usage: 98.0 KB
```

ภาพที่ 29 ชุดข้อมูลคงเหลือหลังจากเพิ่ม Feature Engineering

```
1 NVDIA_fea.describe()
```

✓ 0.0s

	Return	Volatility	Volume_Log	Return_Volume
count	2510.000000	2510.000000	2510.000000	2510.000000
mean	0.002712	0.663480	19.851145	0.054411
std	0.030632	1.187659	0.466715	0.623727
min	-0.187559	0.002303	17.775333	-4.013383
25%	-0.012436	0.060838	19.540827	-0.245944
50%	0.002652	0.198479	19.846311	0.051386
75%	0.017626	0.690138	20.151182	0.350150
max	0.298067	9.076630	22.029685	6.408574

ภาพที่ 30 รายละเอียดค่าทางสถิติเบื้องต้นหลังจากเพิ่ม Feature Engineering

1.3 TSLA Stock

```
1 tesla
```

✓ 0.0s

	Date	Close	High	Low	Open	Volume
0	2015-01-02	14.620667	14.883333	14.217333	14.858000	71466000
1	2015-01-05	14.006000	14.433333	13.810667	14.303333	80527500
2	2015-01-06	14.085333	14.280000	13.614000	14.004000	93928500
3	2015-01-07	14.063333	14.318667	13.985333	14.223333	44526000
4	2015-01-08	14.041333	14.253333	14.000667	14.187333	51637500
...	...	...	...	...	...	...
2511	2024-12-24	462.279999	462.779999	435.140015	435.899994	59551800
2512	2024-12-26	454.130005	465.329987	451.019989	465.160004	76366400
2513	2024-12-27	431.660004	450.000000	426.500000	449.519989	82666800
2514	2024-12-30	417.410004	427.000000	415.750000	419.399994	64941000
2515	2024-12-31	403.839996	427.929993	402.540008	423.790008	76825100

2516 rows x 6 columns

ภาพที่ 31 การนำเข้าข้อมูลของ TSLA Stock

```
1 tesla.info()
```

✓ 0.0s

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2516 entries, 0 to 2515
Data columns (total 6 columns):
#   Column  Non-Null Count  Dtype
---  -
0   Date    2516 non-null   datetime64[ns]
1   Close   2516 non-null   float64
2   High    2516 non-null   float64
3   Low     2516 non-null   float64
4   Open    2516 non-null   float64
5   Volume  2516 non-null   int64
dtypes: datetime64[ns](1), float64(4), int64(1)
memory usage: 118.1 KB
```

ภาพที่ 32 รายละเอียดของชุดข้อมูลดั้งเดิมของ TSLA Stock

```
1 print(tesla.isnull().sum())
2 print("Duplicates:", tesla.duplicated().sum())
```

✓ 0.0s

```
Date      0
Close     0
High      0
Low       0
Open      0
Volume    0
dtype: int64
Duplicates: 0
```

ภาพที่ 33 ตรวจสอบจำนวนค่าว่าง และจำนวนแถวข้อมูลซ้ำกันของ TSLA Stock

```
1 tesla.describe()
```

✓ 0.0s

	Date	Close	High	Low	Open	Volume
count	2516	2516.000000	2516.000000	2516.000000	2516.000000	2.516000e+03
mean	2019-12-31 19:03:31.764706048	115.679423	118.250154	112.986545	115.701436	1.123131e+08
min	2015-01-02 00:00:00	9.578000	10.331333	9.403333	9.488000	1.062000e+07
25%	2017-07-02 06:00:00	17.185167	17.481667	16.889166	17.177167	6.681885e+07
50%	2020-01-01 00:00:00	28.505667	28.806000	27.349999	28.413666	9.284585e+07
75%	2022-06-30 06:00:00	220.205002	225.354996	215.336670	220.925003	1.297886e+08
max	2024-12-31 00:00:00	479.859985	488.540008	457.510010	475.899994	9.140820e+08
std	NaN	114.226440	116.887822	111.485181	114.312114	7.407088e+07

ภาพที่ 34 รายละเอียดค่าทางสถิติเบื้องต้นของ TSLA Stock

```
1 tesla_fea.info()
```

✓ 0.0s

```
<class 'pandas.core.frame.DataFrame'>
Index: 2510 entries, 6 to 2515
Data columns (total 4 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Return          2510 non-null   float64
1   Volatility       2510 non-null   float64
2   Volume_Log      2510 non-null   float64
3   Return_Volume   2510 non-null   float64
dtypes: float64(4)
memory usage: 98.0 KB
```

ภาพที่ 35 ชุดข้อมูลคงเหลือหลังจากเพิ่ม Feature Engineering ของ TSLA Stock

```
1 tesla_fea.describe()
```

✓ 0.0s

	Return	Volatility	Volume_Log	Return_Volume
count	2510.000000	2510.000000	2510.000000	2510.000000
mean	0.001993	4.477440	18.381807	0.038287
std	0.036031	5.635379	0.537851	0.717821
min	-0.210628	0.060867	16.178250	-4.316777
25%	-0.016160	0.419456	18.017601	-0.320749
50%	0.001280	1.452183	18.346753	0.025252
75%	0.019257	7.186524	18.683720	0.381703
max	0.219190	40.597454	20.633431	4.156960

ภาพที่ 36 รายละเอียดค่าทางสถิติเบื้องต้นหลังจากเพิ่ม Feature Engineering ของ TSLA Stock

2. โครงสร้างโค้ด

2.1 โครงสร้างไฟล์และส่วนประกอบสำคัญ

- `main.py`: ไฟล์หลักสำหรับเรียกใช้งานโปรแกรม รับไฟล์ CSV จากผู้ใช้ และประสานงานระหว่างโมดูลต่างๆ (การเตรียมข้อมูล → ตรวจจับ drift → เปรียบเทียบโมเดล → บันทึกผลลัพธ์)
- `data_preparation.py`: จัดการเตรียมข้อมูลและสร้างฟีเจอร์ใหม่ ๆ เช่น Return (ผลตอบแทน), Volatility (ความผันผวน), Volume_Log (ปริมาณการซื้อขาย), และ Return_Volume (ปริมาณการซื้อขายถ่วงน้ำหนักด้วยผลตอบแทน)
- `models.py`: เก็บโมเดลทำนายราคาหุ้น ได้แก่ RNN, LSTM, GRU และ Linear Regression พร้อมระบบจัดการ sequence data และ scaling สำหรับโมเดลแบบ time series
- `drift_detection.py`: ตรวจจับการเปลี่ยนแปลงของข้อมูล (Concept Drift) โดยใช้เทคนิค ADWIN จากไลบรารี river พร้อมกรองจุด drift ที่มีข้อมูลไม่เพียงพอและรวม fold สุดท้ายหากข้อมูลน้อยเกินไป
- `cross_validation.py`: กำหนดกลยุทธ์การแบ่งข้อมูลสำหรับทดสอบโมเดล มีทั้งแบบ Adaptive CV (แบ่งตามจุดที่ข้อมูลเปลี่ยนแปลง) และ Baseline CV (แบ่งแบบ rolling window ทั่วไป) โดยแยกวัดประสิทธิภาพก่อนและหลัง drift
- `model_comparison.py`: เปรียบเทียบผลลัพธ์ของโมเดลทั้งหมด (LSTM, RNN, GRU, Linear) ใน 2 สถานการณ์ (Adaptive และ Baseline) แสดงผลทางหน้าจอและส่งออกเป็นไฟล์ TXT หรือ CSV พร้อมระบุโมเดลที่ดีที่สุดตามค่า RMSE หลัง drift
- `requirements.txt`: ไฟล์สำหรับติดตั้ง dependencies ทั้งหมดของโปรแกรม (numpy, pandas, scikit-learn, tensorflow, river)

2.2 ฟีเจอร์เด่นของโปรแกรม

- การแสดงผลที่ชัดเจน: แสดงผลในรูปแบบตารางที่อ่านง่าย มีการแบ่งส่วนข้อมูลและใช้ emoji เพื่อช่วยให้ดูเข้าใจได้ทันที นอกจากนี้ยังมีการไฮไลต์โมเดลที่ทำนายได้ดีที่สุดด้วย
- การบันทึกผลลัพธ์อัตโนมัติ: สามารถบันทึกผลการเปรียบเทียบโมเดลทั้งหมดลงในไฟล์ .txt โดยอัตโนมัติ ชื่อไฟล์จะมีการใส่ timestamp เพื่อป้องกันชื่อซ้ำ และไฟล์ที่บันทึกไว้จะประกอบด้วยข้อมูลที่ละเอียดสำหรับการทดสอบแต่ละ fold

2.3 รูปแบบไฟล์ CSV ที่รองรับ ไฟล์ข้อมูลจะต้องเป็น ไฟล์ CSV ที่มีคอลัมน์สำคัญดังนี้

- Date (วันที่) - รูปแบบ dd/mm/yyyy
- Close (ราคาปิด)
- Volume (ปริมาณการซื้อขาย)
- High (ราคาสูงสุด) - ถ้ามี
- Low (ราคาต่ำสุด) - ถ้ามี

2.4 โมเดลที่รองรับ โปรแกรมนี้สามารถเปรียบเทียบโมเดลได้ 4 แบบ

- RNN (Simple Recurrent Neural Network)
- LSTM (Long Short-Term Memory)
- GRU (Gated Recurrent Unit)
- LINEAR (Linear Regression)

2.5 คุณสมบัติอื่น ๆ ที่น่าสนใจ

- โปรแกรมจะ ตั้งค่า random seed เพื่อให้ผลลัพธ์คงที่
- TensorFlow ถูกตั้งค่าให้ใช้ single thread เพื่อความเสถียร
- โปรแกรมจะ ข้ามการทดสอบใน fold ที่มีข้อมูลไม่เพียงพอสำหรับ train และ test เพื่อให้การรันโปรแกรมเป็นไปอย่างราบรื่น

3. ขั้นตอนการใช้งานโปรแกรม (Step-by-step)

ขั้นตอนที่ 1: เตรียมไฟล์ข้อมูล

ผู้ใช้งานต้องมีไฟล์ CSV ที่อย่างน้อยมีคอลัมน์ต่อไปนี้

- Date → วันที่
- Close → ราคาปิด
- Volume → ปริมาณการซื้อขาย

ข้อมูลควรเป็น Time Series ต่อเนื่องตามเวลา และมีจำนวนแถวเพียงพอ (แนะนำมากกว่า 100 แถว)

ขั้นตอนที่ 2: ติดตั้งโปรแกรมที่จำเป็น

เปิด Terminal / Command Prompt แล้วรัน

```
pip install -r requirements.txt
```

จากนั้นตรวจสอบว่า

- Python เวอร์ชัน ≥ 3.8

ขั้นตอนที่ 3: รันโปรแกรมหลัก มี 2 วิธี

วิธีที่ 1: ระบุ path ไฟล์ตั้งแต่แรก

```
python main.py "path/to/your/data.csv"
```

วิธีที่ 2: รันเปล่า แล้วป้อน path ที่หลัง

```
python main.py
```

จากนั้นโปรแกรมจะถามให้ใส่ path ของไฟล์ CSV

ขั้นตอนที่ 4: โปรแกรมทำงานอัตโนมัติ (ผู้ใช้ไม่ต้องทำอะไรเพิ่ม)

เมื่อรันแล้ว โปรแกรมจะดำเนินการตามลำดับนี้โดยอัตโนมัติ

1. โหลดข้อมูลจากไฟล์ CSV
2. เตรียมข้อมูลและสร้าง features
3. ตรวจสอบจุดที่ข้อมูลเปลี่ยนพฤติกรรม (Concept Drift)
4. แบ่งข้อมูลตามช่วงเวลา (Adaptive CV และ Baseline CV)
5. เทรนโมเดลหลายแบบ (RNN, LSTM, GRU, Linear Regression)
6. คำนวณค่า RMSE และ MAE
7. เปรียบเทียบประสิทธิภาพของโมเดล

ขั้นตอนที่ 5: ดูผลลัพธ์บนหน้าจอ

หลังประมวลผลเสร็จ โปรแกรมจะแสดงผล เช่น

- จำนวนจุดที่ตรวจพบ concept drift
- ตารางเปรียบเทียบโมเดล
- โมเดลที่มีค่า RMSE เล็กน้อยที่สุด

ผู้ใช้สามารถใช้ผลลัพธ์นี้เพื่อ

- วิเคราะห์ประสิทธิภาพโมเดล
- ใช้ประกอบรายงานหรือการนำเสนอ

ขั้นตอนที่ 6: เลือกบันทึกผลลัพธ์

โปรแกรมจะถามว่า ต้องการบันทึกผลลัพธ์หรือไม่

- กด 1 → บันทึกเป็นไฟล์ .txt (อ่านง่าย เหมาะแนบรายงาน)
- กด 2 → บันทึกเป็นไฟล์ .csv (เหมาะนำไปวิเคราะห์ต่อ เช่น Excel / Tableau)
- กด Enter → ไม่บันทึก

4. วิธีการติดตั้งและรันโค้ด ดาวน์โหลดจาก GitHub

4.1 Clone Repository

- `git clone https://github.com/Prapakorn23/drift-cv-timeseries.git`
- `cd drift-cv- timeseries`

หรือดาวน์โหลด ZIP

- ไปที่ GitHub repository
- คลิก "Code" → "Download ZIP"
- แดกไฟล์และเข้าโฟลเดอร์

4.2. ความต้องการของระบบ

- Python 3.11
- ไลบรารีและแพ็คเกจสำคัญ
 - `numpy>=1.21.0`
 - `pandas>=1.3.0`
 - `tensorflow>=2.8.0`
 - `scikit-learn>=1.0.0`
 - `scipy>=1.7.0`
 - `river>=0.15.0`

4.3 วิธีการใช้งาน คุณสามารถใช้งานโปรแกรมได้ 2 วิธี

- ผ่าน Command Line
 - `python main.py "path/to/your/data.csv"` (ใส่ path ของไฟล์ CSV เป็น argument)
- แบบโต้ตอบ
 - `python main.py` (โปรแกรมจะขึ้น prompt ให้คุณใส่ path ของไฟล์ CSV เอง)